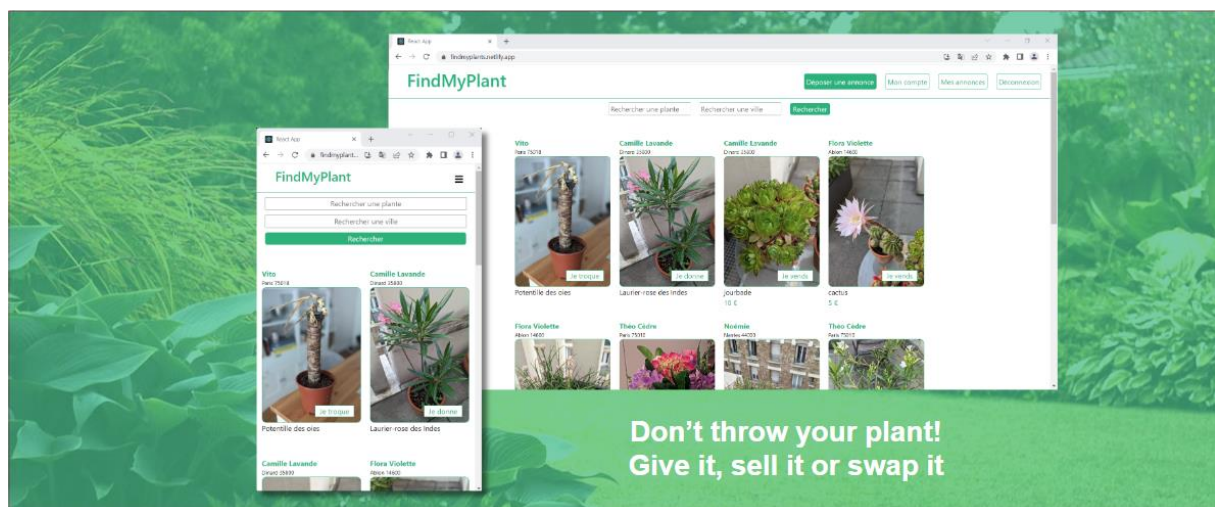


FindMyPlant

Dossier projet

Titre RNCP Développeur web et web mobile



Marc-Antoine VANNIER
03/08/2024

Table des matières

1	Introduction.....	3
2	Liste des compétences mises en œuvre	3
2.1	Compétences techniques	3
2.2	Compétences transversales.....	4
3	Expression des besoins du projet.....	4
3.1	Contexte et objectif	4
3.2	Besoins fonctionnels.....	4
3.2.1	Inscription et gestion des comptes utilisateurs	4
3.2.2	Gestion des annonces	4
3.2.3	Recherche et navigation.....	5
3.2.4	Diagramme des cas d'utilisation	6
3.3	Besoins non fonctionnels.....	6
3.4	Contraintes du projet	7
3.5	Critères de réussite.....	7
3.5.1	Organisation & planning.....	7
4	Environnement technique.....	8
4.1	Langages de programmation utilisés.....	8
4.2	Frameworks et bibliothèques et base de données utilisé(e)s.....	9
4.3	Outils de développement.	9
4.4	Plateformes ou services utilisés.	9
5	Réalisations permettant la mise en œuvre des compétences.....	10
5.1	Réalisation du front-end.....	10
5.1.1	Présentation de maquettes de l'application, adaptation web et adaptation web mobile ; 10	
5.1.2	Schéma de l'enchaînement des maquettes de l'application ;	11
5.1.3	Captures d'écran d'interfaces utilisateur, adaptation web et adaptation web mobile ; 11	
5.1.4	Initialisation du projet	11
5.1.5	Installation des dépendances.....	12
5.1.6	Structure du front-end	12



5.1.7	Les composants	12
5.1.8	Configuration des routes.....	14
5.1.9	Les services.....	15
5.1.10	Extrait de code des certaines pages.....	15
5.1.11	Accessibilité	16
5.1.12	Responsive design	16
5.1.13	Les tests	16
5.1.14	Le déploiement.....	17
5.2	Réalisation du back-end	17
5.2.1	Initialisation du projet	17
5.2.2	Installation des dépendances.....	18
5.2.3	Configuration du serveur Express	19
5.2.4	Présentation de la base de données	19
5.2.5	Création des Routes et Contrôleurs	19
5.2.6	Création des middlewares.....	20
5.2.7	Extraits de code	21
5.2.8	Test de l'API.....	23
5.3	Présentation d'éléments de sécurité de l'application.....	23
5.3.1	Utilisation de variable d'environnement	23
5.3.2	Configuration CORS	24
5.3.3	Sécurités lors de la création d'un compte utilisateur	24
5.3.4	Authentification via Json Web Token.....	26
5.4	Présentation des tests	28
5.5	Description de la veille, vulnérabilités de sécurité.....	32
6	Conclusion	32
7	Annexes	33
8	Table des illustrations	57

1 Introduction

Le projet que je vais présenter s'appelle FindMyPlant. C'est une plateforme web pour tous les passionnés de plantes qui souhaitent donner, vendre/acheter ou troquer des plantes.

Dans le cadre de ma formation à Holberton School, nous avons eu l'opportunité de réaliser un projet de fin d'étude afin de mettre en œuvre les connaissances que nous avons pu acquérir.

Ce qui a rendu ce projet très excitant est la liberté d'utiliser toutes les technologies que nous désirons, et cela sans restriction.

De plus, je voulais créer un projet de bout en bout afin de mieux évaluer et comprendre toutes les complexités qui s'y rattachent. Cela a été une bonne opportunité d'améliorer mes compétences techniques et mes connaissances.

Voici comment m'est venue l'idée de ce projet :

J'ai grandi dans un petit village entouré de champs. Maintenant, j'habite en ville, j'aime avoir des plantes chez moi et prendre soin d'elles. Parfois, je recherche des plantes ou des fleurs qui ne sont pas faciles à trouver localement. Et je me suis dit « Comment rencontrer des gens en ville qui ont des plantes que je recherche ? » Et d'un autre côté, quand j'entretiens mes plantes, je suis parfois obligé d'en jeter une partie à la poubelle, car je n'ai pas un espace infini chez moi. J'aurais pu donner ce surplus à d'autres personnes intéressées.

C'est à ce moment-là que m'est venue l'idée de FindMyPlant. Créer une plateforme simple, où il est facile de trouver des gens qui indiquent clairement s'ils veulent vendre / donner / ou troquer des plantes.

J'ai travaillé seul sur ce projet, mais j'ai fréquemment échangé avec certains camarades de nos projets respectifs car nous faisions face à des problématiques similaires.

2 Liste des compétences mises en œuvre

2.1 Compétences techniques

Développer la partie front-end d'une application web ou web mobile sécurisée :

- Installer et configurer son environnement de travail en fonction du projet web ou web mobile.
- Maquetter des interfaces utilisateur web ou web mobile.
- Réaliser des interfaces utilisateur statiques web ou web mobile.
- Développer la partie dynamique des interfaces utilisateur web ou web mobile.

Développer la partie back-end d'une application web ou web mobile sécurisée :

- Mettre en place une base de données relationnelle.



- Développer des composants d'accès aux données SQL et NoSQL.
- Développer des composants métier coté serveur.
- Documenter le déploiement d'une application dynamique web ou web mobile.

2.2 Compétences transversales

- Communiquer.
- Mettre en œuvre une démarche de résolution de problèmes.
- Apprendre en continu.

3 Expression des besoins du projet

3.1 Contexte et objectif

Le projet consiste à développer une plateforme web dédiée aux passionnés de plantes, permettant aux utilisateurs de publier des annonces pour vendre, donner ou troquer des plantes.

L'objectif principal est de :

- Créer une communauté où les passionnés de plantes peuvent échanger des plantes facilement autour de chez eux.
- Offrir une expérience utilisateur intuitive.
- Faciliter la recherche de plantes.
- Déployer l'application en ligne.
- Avoir une bonne documentation.

3.2 Besoins fonctionnels

3.2.1 Inscription et gestion des comptes utilisateurs

- **Inscription des utilisateurs** : Système de création de compte avec nom de l'utilisateur, email, mot de passe et code postal. Le code postal est nécessaire pour aller récupérer le nom de la ville associée de manière automatique. L'inscription n'est pas obligatoire pour consulter les annonces.
- **Connexion des utilisateurs** : Système de connexion avec l'email et le mot de passe.
- **Profil utilisateur** : Chaque utilisateur doit pouvoir gérer son profil, mettre à jour ses informations ou pouvoir supprimer entièrement son compte.

3.2.2 Gestion des annonces

- **Publication d'annonces** : Les utilisateurs doivent pouvoir publier des annonces et indiquer s'ils souhaitent vendre, donner ou troquer les plantes. Les annonces incluront des informations telles que :
 - Le nom de la plante.



- Les conditions (Je vends, Je donne, Je troque).
 - Les prix (si applicable).
 - Commentaires
 - Ajouter des images
- **Gestion des annonces** : Les utilisateurs doivent pouvoir modifier, supprimer ou mettre à jour leurs annonces. Mais également rendre une annonce non disponible sans pour autant la supprimer.

3.2.3 Recherche et navigation

- **Recherche d'annonces** : Les utilisateurs doivent pouvoir filtrer les annonces par nom de plante ou nom de la ville.
- **Aide à la recherche** : A mesure que l'utilisateur entrera le nom d'une plante, une liste déroulante doit apparaître avec le nom final des plantes existantes issue d'une base donnée de plusieurs milliers de plantes.

3.2.4 Diagramme des cas d'utilisation

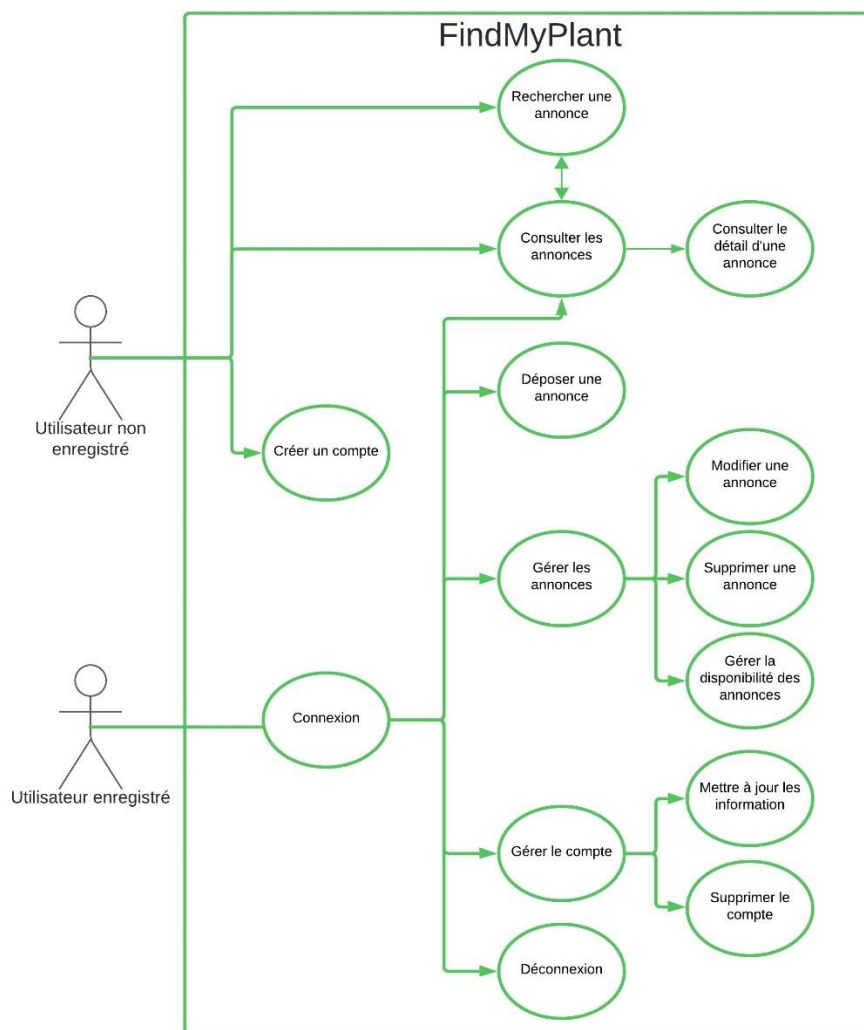


Figure 3-1 Diagramme des cas d'utilisations

3.3 Besoins non fonctionnels

Voici les besoins non fonctionnels que j'ai identifiés.

- Sécurité des données des utilisateurs.
- Performance du site web.
- Interface utilisateur : Design simple et intuitif, adapté à tous les types d'utilisateurs, avec une navigation fluide et des sections bien organisées.
- Responsive design : Le site doit être accessible et utilisable sur tous types d'appareils (ordinateurs, tablettes, smartphones).
- Accessible en ligne.

3.4 Contraintes du projet

Pour la réalisation de ce projet j'ai décidé de partir sur des technologies que je ne connaissais pas afin d'améliorer mes connaissances. Je rentrerai plus en détails sur ces éléments dans la partie **IV. Environnement technique**. Cela était un risque puisque j'ai dû apprendre à utiliser NodeJS, REACT et MongoDB avant et pendant le développement de mon projet. De plus, comme je ne connaissais pas leur degré de complexité, cela a rendu la planification des étapes du projet plus complexe.

Malgré le fait que j'ai travaillé seul sur ce projet, je me suis mis dans les mêmes conditions pour la réalisation d'un projet en équipe. Notamment via la définition des tâches et du délai de chaque tâche sur Trello et l'utilisation de plusieurs branches lors du développement de l'application via Git.

3.5 Critères de réussite

Pour la réussite de ce projet, je m'étais fixé comme objectif :

- Que l'application soit fonctionnelle.
- Qu'elle soit accessible sur les principaux navigateurs.
- Qu'elle soit accessible sur tout type d'appareil.
- Qu'elle soit déployée en ligne.

3.5.1 Organisation & planning

Comme indiqué précédemment, j'ai utilisé Trello pour suivre la progression de mon projet.

J'ai découpé mon projet en une multitude de petites tâches en estimant la durée nécessaire pour les réaliser.

Le délai pour la réalisation de la première version de mon projet a été de 3 semaines pour présenter une application fonctionnelle à la fin de ma première année. Puis j'ai eu un délai de 2 semaines à la fin de ma spécialisation back-end où j'ai pu rajouter de nouvelles fonctionnalités et travailler sur l'intégration et le déploiement continu de mon application.



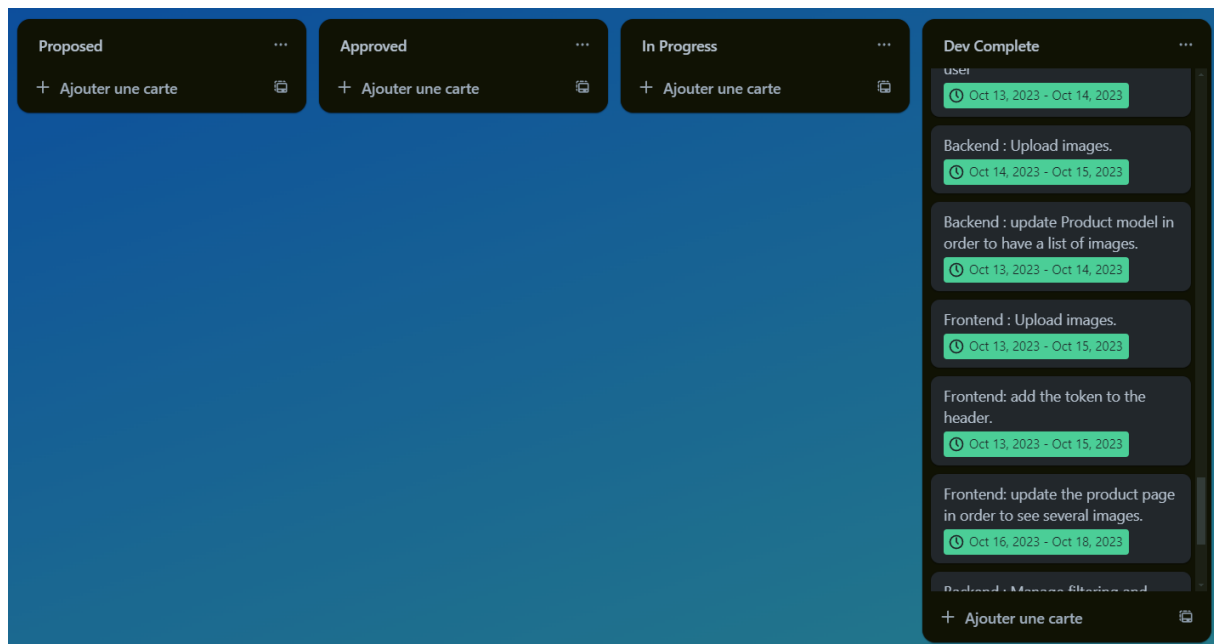


Figure 3-2 Organisation et planning avec Trello

4 Environnement technique

Voici un schéma montrant l'architecture globale ainsi que les technologies utilisées.

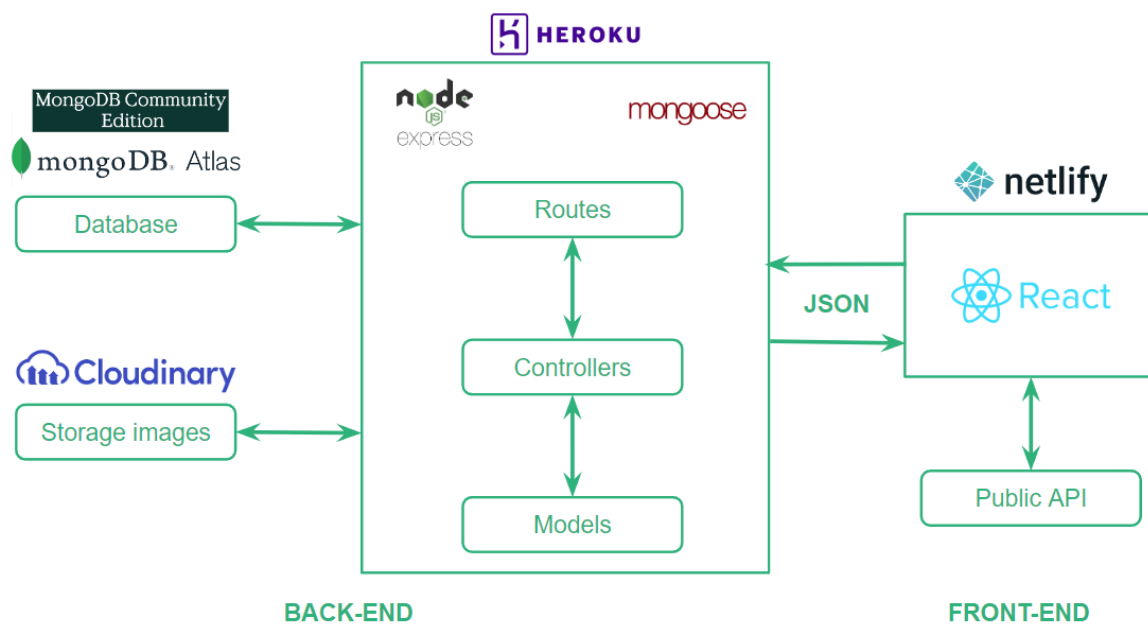


Figure 4-1: Architecture et technologie

4.1 Langages de programmation utilisés.

Lors de la formation, nous avons principalement travaillé les langages de programmations suivants : le C, le Python, puis le Javascript.

Pour mon projet de fin d'étude, je voulais travailler sur de nouvelles technologies et améliorer mes connaissances sur le Javascript. C'est pourquoi j'ai choisi ce langage.

4.2 Frameworks et bibliothèques et base de données utilisé(e)s.

Pour le back-end, j'ai choisi les frameworks suivant : NodeJS et Express.

Pour le front-end, j'ai choisi la bibliothèque suivante : REACT.

Pour la base de données, nous avons principalement travaillé sur du MySQL lors de la formation. Pour ce projet, j'ai choisi d'utiliser MongoDB (no SQL) car cela permet d'avoir plus de flexibilité sur le schéma.

4.3 Outils de développement.

Voici les différents outils que j'ai installés pour le développement :

- **Visual Studio Code**, pour l'éditeur de code.
- **Git**, pour l'outil de gestion des versions. Dans le cadre de ce projet, j'ai utilisé deux branches :
 - o **Branche « dev »** : Toutes les nouvelles fonctionnalités sont développées sur cette branche. Un fois que le code a passé les tests, j'ai effectué des « pull request » de la branche de développement vers la branche « main ».
 - o **Branche « main »** : La branche principale accueille le code qui ira en production.
- **Docker**, pour la virtualisation de mon espace de développement. L'intérêt est que je n'avais pas besoin d'installer de dépendance sur ma machine en local. Donc en cas de panne de l'ordinateur, il aurait été assez rapide de continuer le développement sur un autre ordinateur sans avoir besoin de tout reparamétrer.

4.4 Plateformes ou services utilisés.

Voici les plateformes et services utilisés :

- **Github** : Pour l'hébergement (sauvegarde) du code source de mon application. Utilisé également pour merger ma branche de développement à ma branche principale.
- **Github actions** : Pour l'intégration et le déploiement continu. Cela permettait de tester mon application de manière continue à chaque fois que j'envoyais du nouveau code sur Github. Il y a trois workflows de configurés, un pour le front-end, un pour le back-end et un pour le déploiement du back-end sur Heroku.
- **Heroku** : Pour le déploiement du back-end et accessible en ligne.
- **Netlify** : pour le déploiement du front-end et accessible en ligne.
- **MongoDB Atlas** : Pour la sauvegarde et l'accès de ma base de données en ligne.
- **Cloudinary** : Pour l'hébergement et le traitement des images, accessible en ligne.
- **geo.api.gouv.fr** : J'ai utilisé cette API publique afin de pouvoir récupérer le nom d'une commune grâce au code postal.



5 Réalisations permettant la mise en œuvre des compétences

5.1 Réalisation du front-end

Pour le front-end, j'ai utilisé React car il est bien connu pour sa réactivité grâce à son DOM virtuel et permet une mise à jour rapide de l'interface utilisateur. De plus, nous pouvons créer des composants modulaires et réutilisables, ce qui permet d'avoir un code plus propre.

Il y a une large communauté sur React, ce qui m'a été d'une grande aide pour monter en compétence.

5.1.1 Présentation de maquettes de l'application, adaptation web et adaptation web mobile ;

Avant de passer au développement de mon application, j'ai réalisé les maquettes. Pour cela, j'ai utilisé figma et Basalmiq.

Voici la description des maquettes :

- **Page d'accueil (Home)** : Contient la barre de navigation et la liste de toutes les plantes. (Voir Annexe 1)
- **Page de Connexion (Login)** : Formulaire pour connecter un utilisateur. (Voir Annexe 2)
- **Page S'enregistrer (Sign Up)** : Formulaire pour créer un nouvel utilisateur. (Voir Annexe 3)
- **Page Déposer une Annonce (Create Ad)** : Formulaire pour ajouter une nouvelle plante. (Voir Annexe 4)
- **Page Mon compte** : Page où un utilisateur peut voir et modifier ses informations personnelles. (Voir Annexe 5)
- **Page Mes annonces** : Permet à l'utilisateur de gérer les plantes qu'il a ajoutées. (Voir Annexe 6)
- **Page Modifier une annonce** : Permet à l'utilisateur de modifier les informations d'une plante qu'il a déjà ajoutées. (Voir Annexe 7)
- **Page de Détails de la Plante** : Affiche toutes les informations sur une plante spécifique et permet de contacter le créateur de l'annonce par email. (Voir Annexe 8)

5.1.2 Schéma de l'enchaînement des maquettes de l'application ;

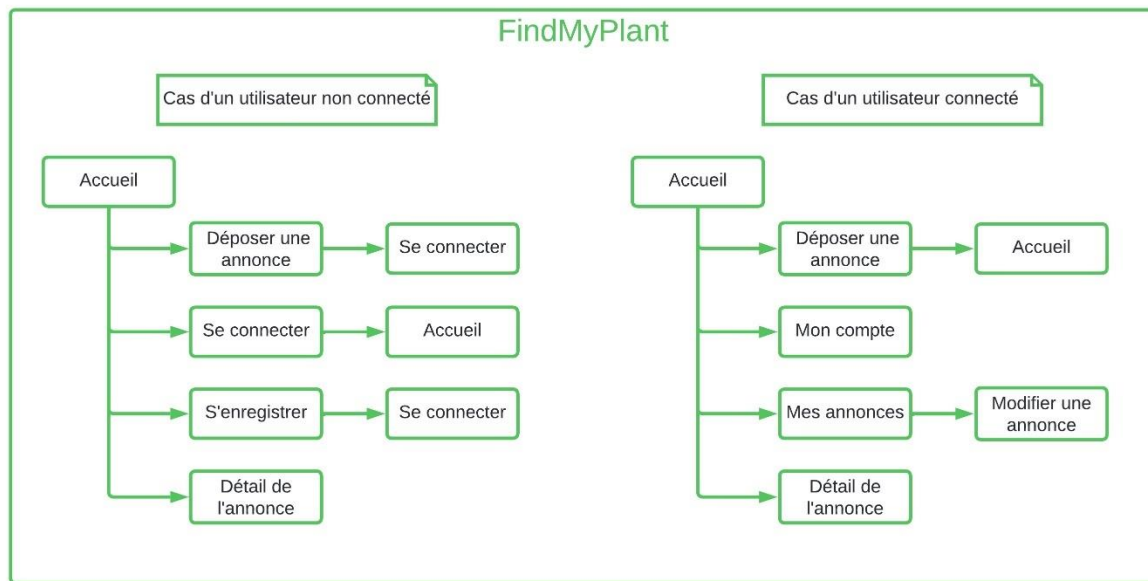


Figure 5-1 Schéma d'enchaînement des maquettes

5.1.3 Captures d'écran d'interfaces utilisateur, adaptation web et adaptation web mobile ;

- **Page d'accueil (Home)** : (Voir Annexe 9)
- **Page de Connexion (Login)** : (Voir Annexe 10)
- **Page S'enregistrer (Sign Up)** : (Voir Annexe 11)
- **Page Déposer une Annonce (Create Ad)** : (Voir Annexe 12)
- **Page Mon compte (Account)** : (Voir Annexe 13)
- **Page Mes annonces (Account Products)** : (Voir Annexe 14)
- **Page Modifier une annonce (Modify Ad)** : (Similaire à l'Annexe 12)
- **Page de Détails de la Plante (Product Detail)** : (Voir Annexe 15)
- **Page 404 (not Found)** : (Voir Annexe 16)

5.1.4 Initialisation du projet

J'ai utilisé Docker pour initialiser mon projet afin de ne pas installer NodeJS et npm en local sur ma machine.

Pour cela j'ai créé un dockerfile nommé « node-utils ».

```
FROM node:18-alpine
WORKDIR /app
```

Puis pour initialiser le projet et enregistrer mon projet sur machine en local j'ai exécuté la commande suivante dans mon dossier « findmyplant » :

```
docker run -it --rm -v $PWD:/app node-utils npx create-react-app frontend
```

5.1.5 Installation des dépendances

Voici la liste des dépendances installées pour la réalisation du front-end.

- @testing-library/jest-dom : Assertions personnalisées pour tester le DOM avec Jest.
- @testing-library/react : Utilitaires pour tester des composants React.
- @testing-library/user-event : Simule les interactions utilisateur pour les tests.
- axios : Client HTTP pour faire des requêtes API.
- bootstrap : Framework CSS pour créer des interfaces responsives.
- jwt-decode : Décode les JSON Web Tokens (JWT).
- moment : Manipulation et formatage des dates.
- react : Bibliothèque JavaScript pour créer des interfaces utilisateur.
- react-bootstrap : Composants Bootstrap pour React.
- react-dom : Rend React utilisable dans le navigateur.
- react-hook-form : Gestion des formulaires avec hooks dans React.
- react-icons : Collection d'icônes pour React.
- react-router-dom : Routage et navigation pour les applications React.
- react-scripts : Scripts et configurations pour créer des applications React.
- web-vitals : Mesure les métriques de performance essentielles du web.

L'installation des dépendances a également été effectuée avec le dockerfile « node-utils ».

Voici un exemple de commande exécuté dans mon dossier « front-end » pour installer jwt-decode :

```
docker run -it --rm -v $PWD:/app node-utils npm i jwt-decode
```

5.1.6 Structure du front-end

La structure du front-end se décompose comme suit :

- **src** : Contient tout le code source de l'application.
 - **components** : Contient les composants React.
 - **styles** : Contient fichiers CSS.
 - **services** : Contient le service lié à la gestion du token et de l'identifiant utilisateur dans la local storage.
 - **Views** : Contient le visuel des différentes pages.

5.1.7 Les composants

Voici un schéma permettant la visualisation des composants sur la page d'accueil.



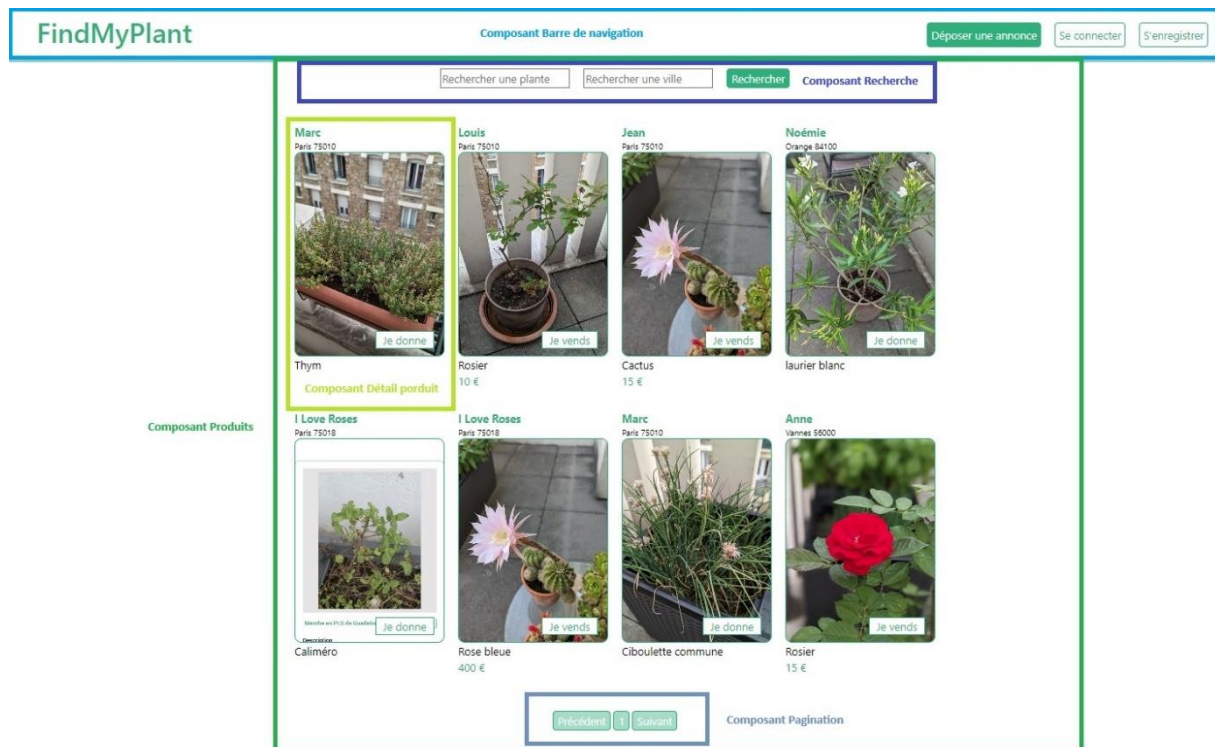


Figure 5-2 Schéma de représentation des composants React

Ci-dessous se trouvent quelques explications concernant les composants mentionnés dans le schéma.

- « **Barre de navigation** » retourne :
 - Les titre de l'application.
 - Un bouton « déposer une annonce ».

Si l'utilisateur n'est pas connecté :

- Un bouton « Connexion », lien vers la page de connexion.
- Un bouton « S'enregistrer », lien vers la page d'inscription.

Si l'utilisateur est connecté :

- Un bouton « Mon compte », lien vers une future page de compte.
- Un bouton « Mes annonces » : lien viens la page Mon compte.
- Un bouton « Déconnexion » : Au clic, ce bouton utilise une fonction qui supprime le token du local storage.
- « **Produits** » récupère tous les produits et retourne une liste de plantes.
- « **Détail Produit** » affiche les différents éléments d'une annonce.
- « **Recherche** » permet d'effectuer une recherche par « nom de plante » ou par « nom de ville ». Lors d'une recherche par « nom de plante », une liste déroulante s'affiche proposant des noms de plantes existantes. Voici une illustration :

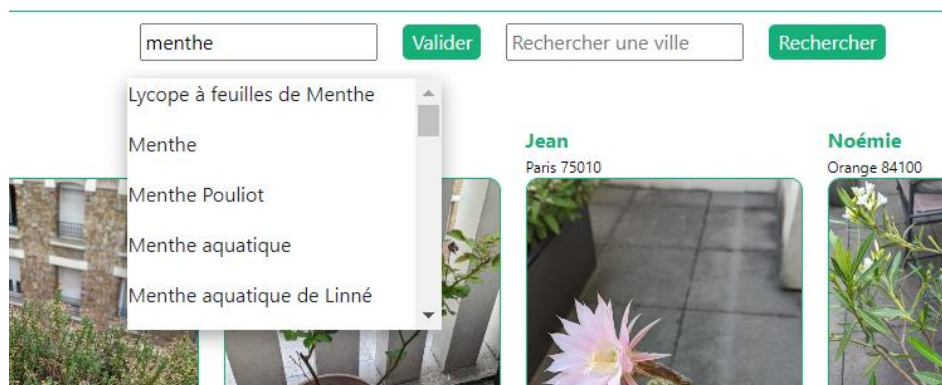


Figure 5-3 Barre de recherche avec liste déroulante

- « **Pagination** » indique le nombre page existantes.

Voici les autres composants :

- « **Routes authentifiées** » vérifie si un utilisateur est bien connecté pour accéder à une page spécifique. Pour cela il s'assure que le token d'authentification est existant dans le local storage. Il decode le token grâce au module « `jwt_decode` » et vérifie que le token est toujours valide grâce à la date d'expiration.
- « **app** » gère l'ensemble des routes du front-end.

5.1.8 Configuration des routes

Les routes du front-end sont définies dans le composant « app » grâce à la librairie « react-router-dom ». (Voir annexe 17)

Ci-dessous ce trouve un schéma illustrant les routes du front-end.

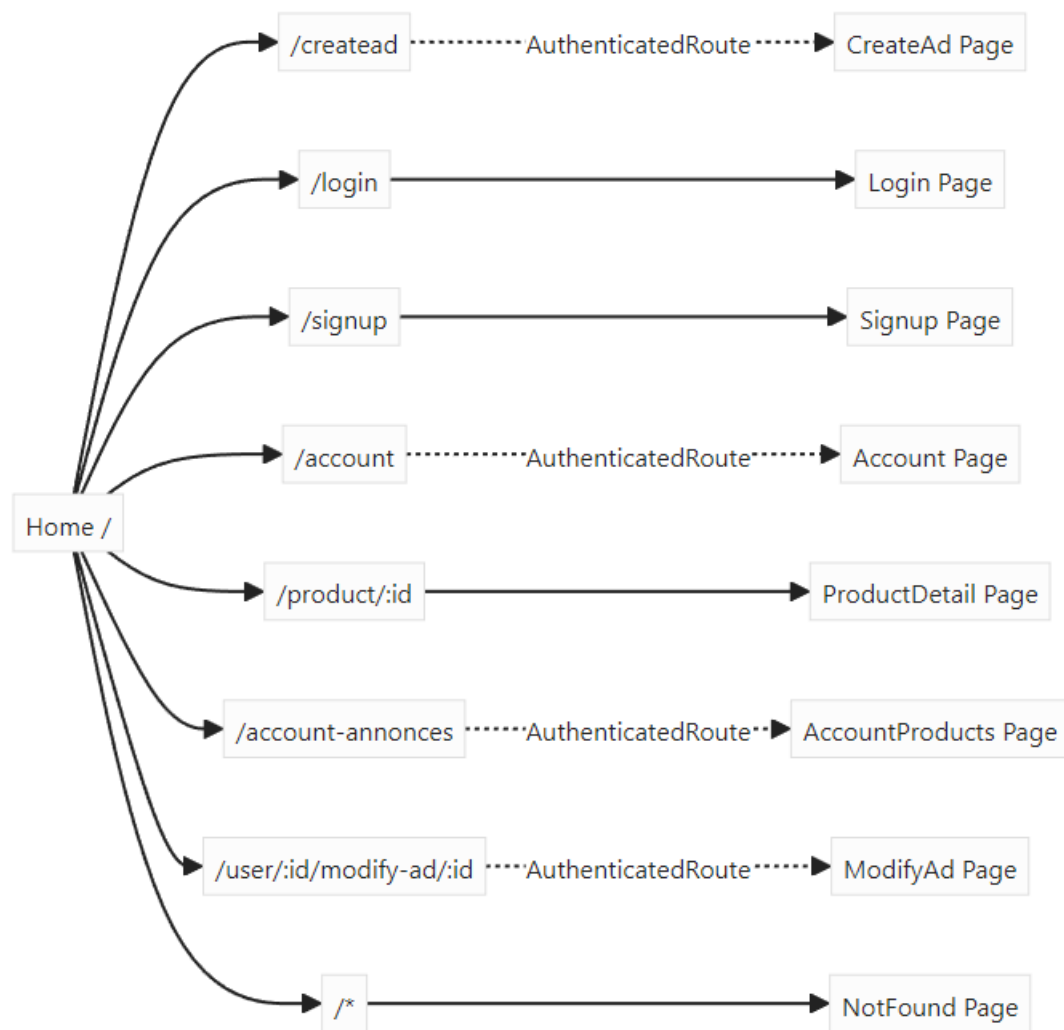


Figure 5-4 Flowchart des routes front-end

5.1.9 Les services

Dans un sous dossier « services », j'ai créé un fichier « accountServices » qui s'occupe de la gestion du token et de l'identifiant utilisateur dans le local storage. (Voir annexe 18)

- `saveToken(token)` : Enregistre le token, le UserId dans le local storage.
- `getToken()` : Récupère le UserId du local storage.
- `logout()` : Supprime le token du local storage, utilisé par le bouton « Déconnexion ».
- `IsLogged()` : Récupère le token du local storage et retourne une valeur booléenne.

5.1.10 Extrait de code des certaines pages

Je présente dans cette partie, le code concernant les pages suivantes :

- **Page d'accueil (Home)** : Dans page d'accueil, nous retrouvons bien le module « barre de navigation » ainsi que le module « produit » présenté précédemment. (Voir Annexe 19)
- **Page Déposer une Annonce (Create Ad)** : (Voir Annexe 20)
- **Page Mes annonces (Account Products)** : (Voir Annexe 21)

5.1.11 Accessibilité

A ce jour, il n'y a pas eu d'action particulière sur l'accessibilité. Après quelques recherches il faudrait intégrer les attributs ARIA (Accessible Rich Internet Applications) qui fournissent des informations supplémentaires aux technologies d'assistance.

5.1.12 Responsive design

Pour appliquer le « Responsive Design » à mon application, j'ai appliqué la règle « @media » dans mes fichiers CSS pour l'ensemble des pages.

Voici un exemple pour la barre de navigation illustré en Annexe 22.

5.1.13 Les tests

Pour vérifier le bon fonctionnement de la partie front-end, j'ai réalisé les tests manuellement au fur et à mesure que je codais.

Néanmoins, j'ai mis en place un workflow avec github action qui s'exécute dès que j'exécute la commande « git push » sur ma branche de développement et également quand il y a une « pull request » de cette dernière branche vers la branche « main ».

Dans ce workflow j'exécute les commandes suivantes :

```
- run: npm ci
- run: npm run lint
- run: npm run build
```

- **npm ci** : Cela permet d'installer les dépendances du projet. Cette commande permet d'utiliser le fichier « package-lock.json » ce qui permet de garantir que les mêmes versions des dépendances sont installées chaque fois.
- **npm run lint** : Cela exécute ESLint qui est un outil d'analyse de code statique. ESLint permet de maintenir un style de code cohérent et détecte des erreurs potentielles avant l'exécution du code.
- **Npm run build** : Cela permet de compiler l'application pour la production.



5.1.14 Le déploiement

Comme indiqué précédemment, le front-end de l'application a été déployé sur Netlify.

Le paramétrage du déploiement s'effectue directement sur Netlify. Dès qu'il y a une « pull request » de la branche de développement vers la branche « main », l'application est automatiquement déployée sur la plateforme comme illustré ci-dessous.

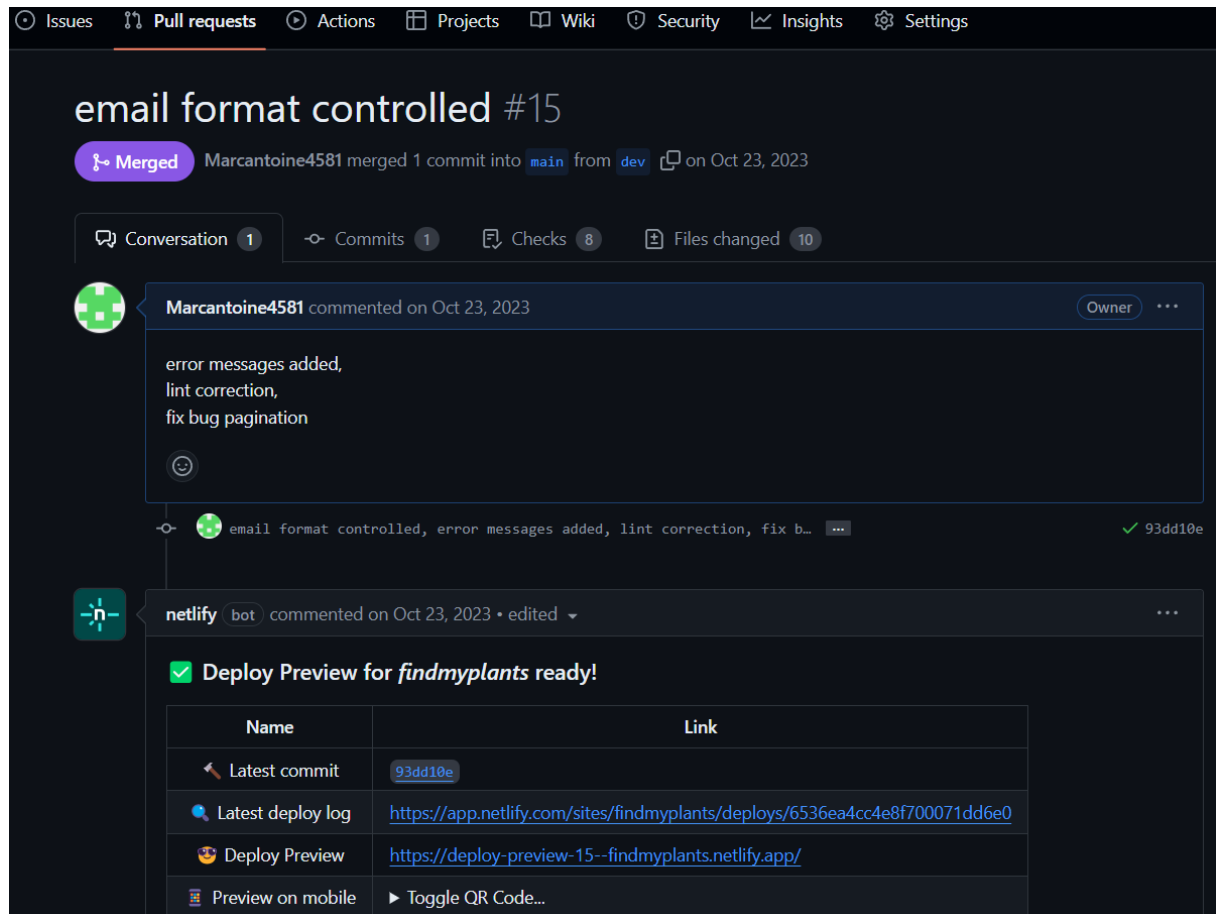


Figure 5-5 Front-end - déploiement sur Netlify

5.2 Réalisation du back-end

Pour la réalisation du back-end, j'ai utilisé le framework NodeJS et Express pour la création de mon application et des API REST.

Les différentes étapes / éléments qui composent ce dernier sont illustrés dans les parties suivantes.

5.2.1 Initialisation du projet

J'ai utilisé Docker pour initialiser mon projet afin de ne pas installer NodeJS et npm en local sur ma machine.

Pour cela j'ai créé un dockerfile nommé « node-utils ».

```
FROM node:18-alpine  
WORKDIR /app
```

Puis pour initialiser le projet et enregistrer mon projet sur machine en local j'ai exécuté la commande suivante dans mon dossier « backend » :

```
docker run -it --rm -v $PWD:/app node-utils npm init
```

5.2.2 Installation des dépendances

Voici la liste des dépendances installées pour la réalisation du back-end.

- **bcrypt** : Une bibliothèque pour hacher et comparer les mots de passe.
- **cloudinary** : Un service cloud pour gérer et manipuler les médias.
- **dotenv** : Charge les variables d'environnement à partir d'un fichier `.env` dans `process.env`.
- **email-validator** : Valide les adresses email.
- **express** : Un framework minimaliste pour construire des applications web et des API.
- **jsonwebtoken** : Génère et vérifie les JSON Web Token (JWT) pour l'authentification.
- **mongoose** : ODM (Object Data Modeling) pour MongoDB et Node.js, facilite l'interaction avec MongoDB.
- **mongoose-unique-validator** : Plugin Mongoose pour valider l'unicité des champs dans les schémas.
- **morgan** : Middleware de logging HTTP pour Node.js.
- **multer** : Middleware pour gérer les uploads de fichiers multipart/form-data.

Voici la liste des dépendances de développement.

- **chai** : Une bibliothèque d'assertions pour les tests.
- **chai-http** : Plugin Chai pour tester les requêtes HTTP.
- **eslint** : Un outil pour analyser et faire respecter des règles de style de code.
- **eslint-config-airbnb** : Configuration ESLint basée sur les règles de style Airbnb.
- **eslint-config-airbnb-base** : Base de la configuration ESLint de style Airbnb sans React.
- **mocha** : Un framework de test pour Node.js.
- **nodemon** : Outil pour redémarrer automatiquement le serveur lors des changements de fichiers.
- **prettier** : Un formateur de code pour assurer un style de code cohérent.

L'installation des dépendances a également été effectuée avec le dockerfile « node-utils ».

Voici un exemple de commande pour installer express :

```
docker run -it --rm -v $PWD:/app node-utils npm i express
```



5.2.3 Configuration du serveur Express

Pour la configuration du serveur Express, j'ai créé un fichier « server.js ». (Voir Annexe 23)

Dans la configuration, nous trouvons également les éléments suivants :

- Gestion des en-têtes Cross-Origin Resource Sharing (CORS) : Ce middleware est exécuté pour chaque requête entrante afin de débloquent les appels HTTP entre différents serveurs.
- Connexion à MongoDB.
- L'accès aux routes.
- L'utilisation du middleware Morgan de logging HTTP.

5.2.4 Présentation de la base de données

Voici comment a été conçue la base de données.

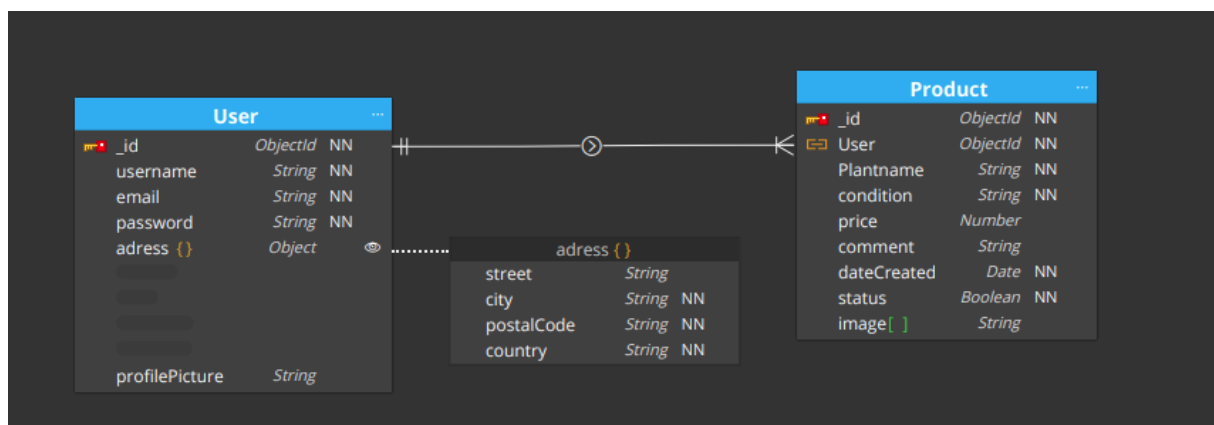


Figure 5-6 Schéma de la base de données

Dans mon back-end, les modèles de données ont été créés avec Mongoose.

Dans un sous-dossier « models », j'ai créé deux fichiers comme suit :

- « User.js » pour le schéma lié aux informations d'utilisateur. (Voir Annexe 24)
- « Product.js » pour le schéma lié aux informations d'un produits et contenant un ID utilisateur. (Voir Annexe 25)

5.2.5 Création des Routes et Contrôleurs

Voici une représentation générale des classes pour les contrôleurs.

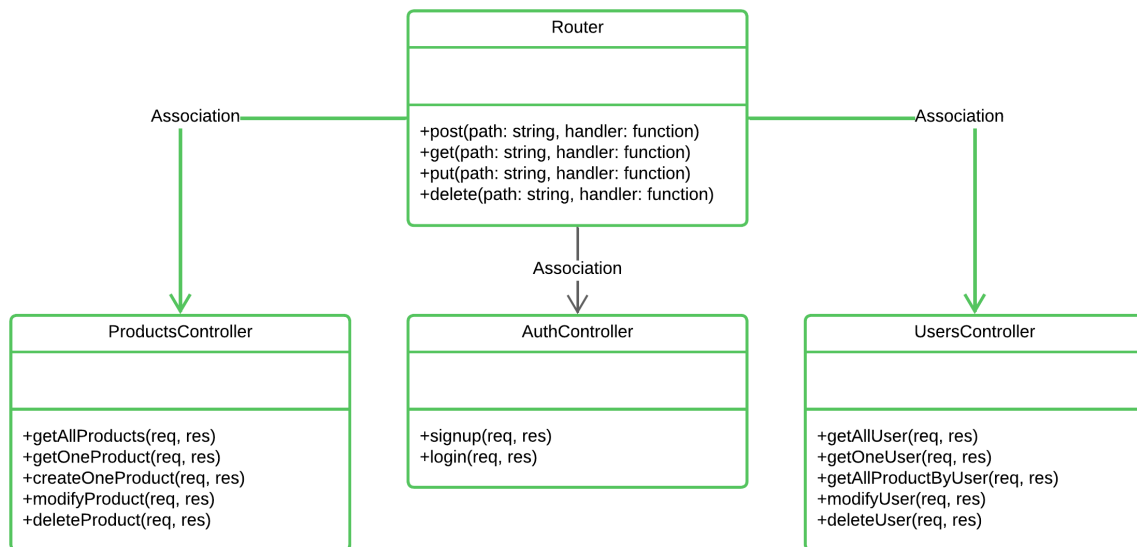


Figure 5-7 Diagramme de classe UML (Routes et Contrôleurs)

J'ai dans un premier temps créé mes routes API comme suit :

- Routes liées à l'authentification :
 - **POST api/auth/signup** - Crée un nouvel utilisateur.
 - **POST /api/auth/login** - Authentifie un utilisateur en fonction de l'email et du mot de passe fournis.
- Routes liées aux utilisateurs :
 - **GET api/users** - Renvoie une liste de tous les utilisateurs.
 - **GET api/users/:id** - Renvoie l'utilisateur avec l'ID spécifié.
 - **GET /api/users/:id/products** - Renvoie tous les articles publiés par un utilisateur.
 - **PUT api/users/:id** - Met à jour les informations d'un utilisateur avec l'ID spécifié.
 - **DELETE api/users/:id** - Supprime l'utilisateur avec l'ID spécifié et tous les produits le concernant.
- Routes liées aux produits :
 - **GET api/products** - Renvoie une liste de tous les produits ou des produits liés à une recherche.
 - **GET api/products/:id** - Renvoie le produit avec l'ID spécifié.
 - **POST api/products** - Crée un nouveau produit.
 - **PUT api/products/:id** - Met à jour le produit avec l'ID spécifié.
 - **DELETE api/products/:id** - Supprime le produit avec l'ID spécifié.

5.2.6 Création des middlewares

Les middlewares mis en place permettent d'intercepter et modifier les requêtes entrantes avant qu'elles ne parviennent aux gestionnaires de routes.

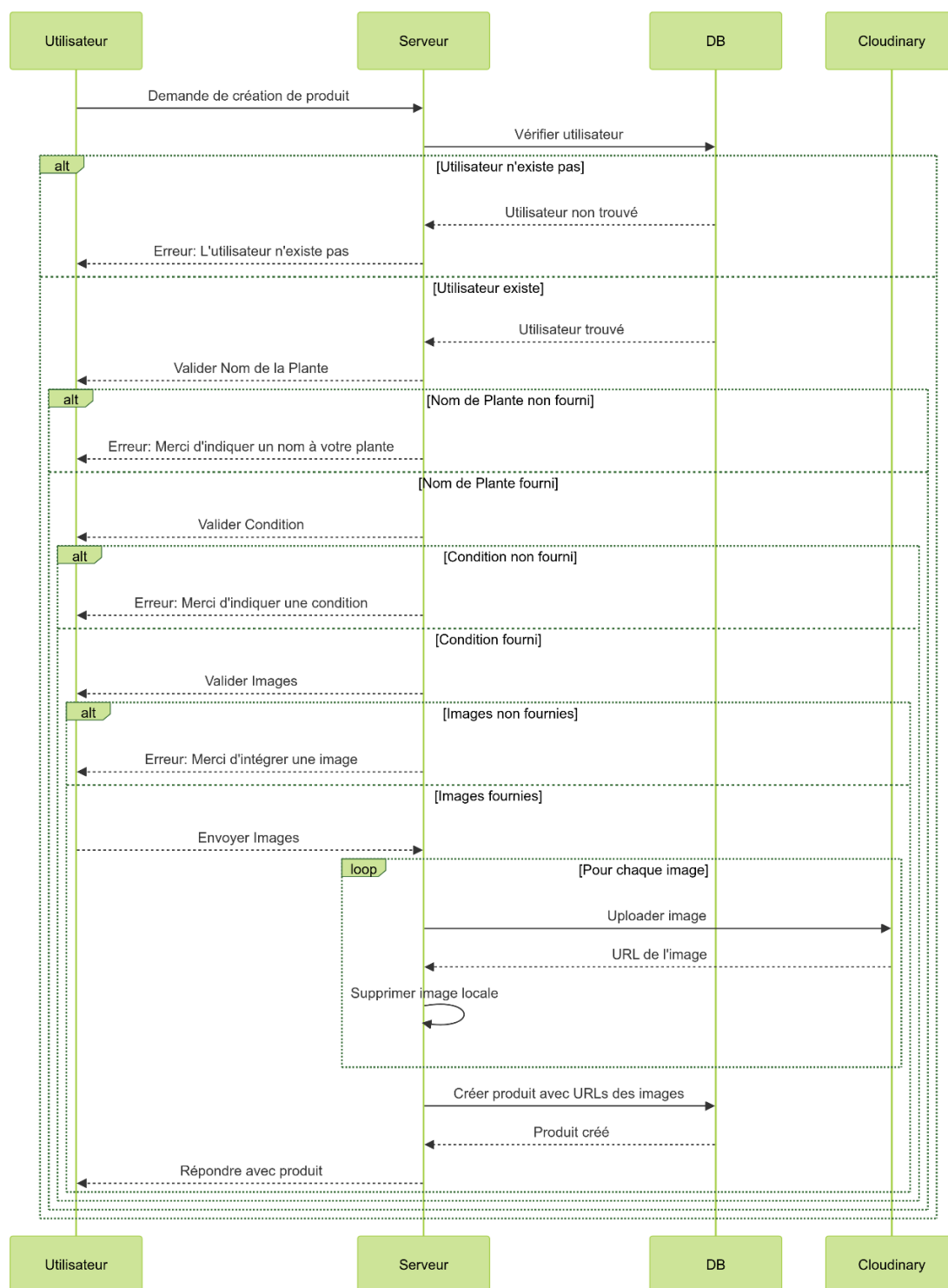
- **Authentication** : Ce middleware est détaillé dans la partie « 5.3.1 Authentification via Json Web Token »
- **La pagination** : Ce middleware permet d'éviter de renvoyer au front-end une trop grande quantité de donnée mais uniquement ce dont il a besoin. Il récupère de la requête le numéro de la page ainsi que les éléments de recherche d'un utilisateur (le nom d'une plante et/ou la ville). Suite à cela il retourne dans la réponse un nombre défini d'annonces. (Voir annexe 26)
- **Gestion des images** : Multer est un middleware pour gérer les fichiers multipart/form-data, principalement utilisé pour télécharger des fichiers dans Node.js. Ce middleware indique l'emplacement où l'image sera téléchargée et la renomme afin de s'assurer que toutes les images ont un nom unique. (Voir Annexe 27)

5.2.7 Extraits de code

Dans cette partie, voici les extraits de code des composants métiers suivants :

- **Création d'une annonce** (Voir Annexe 28) :
 - Récupération des éléments suivants issue du corps de la requête : Identifiant de l'utilisateur, Nom de la plante, Conditions (Je donne, je vends, je troque)
 - Vérification si les éléments ci-dessus sont bien présents sinon on retourne le code de statut de réponse HTTP « 400 Bad Request ».
 - Récupération des images.
 - Vérification qu'il y a au moins une image sinon on retourne le code de statut de réponse HTTP « 400 Bad Request ».
 - Transfert des images sur Cloudinary puis retourne l'URL des images.
 - Suppression des images sur le serveur back-end.
 - Sauvegarde des informations dans la base de données.
 - Retourne le code de statut de réponse HTTP « 201 Created ».
 - Si une étape échoue depuis la récupération des images, la fonction retourne le code de statut de réponse HTTP « 500 Internal Server Error ».





- **Suppression d'un compte utilisateur** (voir Annexe 29)
 - Récupération de l'identifiant de l'utilisateur depuis les paramètres de la requête.
 - Vérification si l'utilisateur existe bien en base de données sinon la fonction retourne le code de statut de réponse HTTP « 400 Bad Request ».
 - Récupération de toutes annonces créées pour l'utilisateur via une requête en base de données.



- Suppression des images dans Cloudinary.
- Suppression de toutes annonces créées pour l'utilisateur dans la base de données.
- Suppression des informations de l'utilisateur.

5.2.8 Test de l'API

Pour tester mon API manuellement, j'ai utilisé POSTMAN.

La documentation est accessible en ligne à l'adresse suivante : <https://documenter.getpostman.com/view/27249451/2s9YRB1X3i>

Voici un exemple pour la modification des données d'un utilisateur.

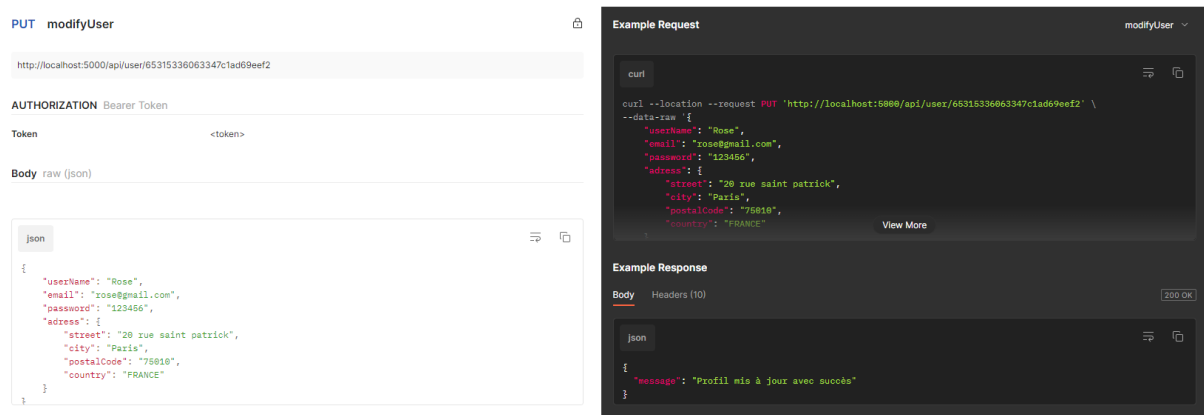


Figure 5-8 Documentation API dans Postman (Modification des données d'un utilisateur)

Un jeu de test est également présenté dans la partie 5.4.

5.3 Présentation d'éléments de sécurité de l'application

Afin d'améliorer la sécurité, voici les éléments que j'ai mis en place.

5.3.1 Utilisation de variable d'environnement

La mise en place de variable d'environnement est également essentielle pour la sécurisation d'une application web.

Les variables d'environnement ont été créées dans un fichier «.env» pour les front-end et le back-end.

Voici la liste des variables d'environnement :

Pour le back-end :

```
# database
MONGODB_URI_DEV = mongodb://127.0.0.1:27017/test
MONGODB_URI_PROD = <MONGODB_URI_PROD>

# JsonWebToken
SECRET_TOKEN = this_is_a_secret_key
```




```
# Cloudinary (storage for images)
CLOUD_NAME = <provided by cloudinary>
CLOUD_KEY = <provided by cloudinary>
CLOUD_SECRET_KEY = <provided by cloudinary>

# Access-Control-Allow-Origin
HTTP_ORIGIN_DEV = http://localhost:3000
HTTP_ORIGIN_PROD = <HTTP_ORIGIN_PROD>
```

Pour le front-end :

```
REACT_APP_API_URL=http://localhost:5000
REACT_APP_END_POINT_PRODUCTS=/api/products/
REACT_APP_END_POINT_USER=/api/user/
REACT_APP_END_POINT_AUTH=/api/auth/
```

5.3.2 Configuration CORS

Dans un premier temps j'ai configuré les en-têtes CORS (Cross-Origin Resource Sharing) afin de restreindre l'accès au back-end uniquement à l'adresse html du front-end. Cela permet de rendre plus difficile pour les attaquants d'exploiter des requêtes provenant de sites non autorisés.

```
const HTTP_ORIGIN = process.env.NODE_ENV === 'production' ? process.env.HTTP_ORIGIN_PROD :
process.env.HTTP_ORIGIN_DEV;

app.use((req, res, next) => {
  res.setHeader('Access-Control-Allow-Origin', `${HTTP_ORIGIN}`);
  res.setHeader('Access-Control-Allow-Headers', 'Origin, Content-Type, Authorization');
  res.setHeader('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE');
  next();
});
```

5.3.3 Sécurités lors de la création d'un compte utilisateur

Lors de la création d'un compte, un contrôle des données entrées par l'utilisateur est effectué via le front-end et également par le back-end.

J'ai effectué cette double vérification afin de limiter les requêtes au serveur back-end et ainsi améliorer les performances.

Le front-end vérifie qu'il n'y a pas d'informations manquantes et que le mot de passe renseigné ainsi que la confirmation de ce dernier correspondent bien.

Le front-end s'assure également de récupérer un nom de ville suite au code postal renseigné par l'utilisateur. Cela m'assure qu'une ville est toujours renseignée dans une annonce.

Le back-end vérifie que l'email renseigné est valide grâce à la librairie « email-validator » et contrôle que cet email n'est pas déjà utilisé par un autre utilisateur.

Le mot de passe est crypté via la librairie « bcrypt » avant d'être sauvegardé dans la base de données.



L'utilisation de brycpt permet de limiter les attaques brute force.

Vous trouverez ci-dessous un diagramme de séquence de la création de compte.

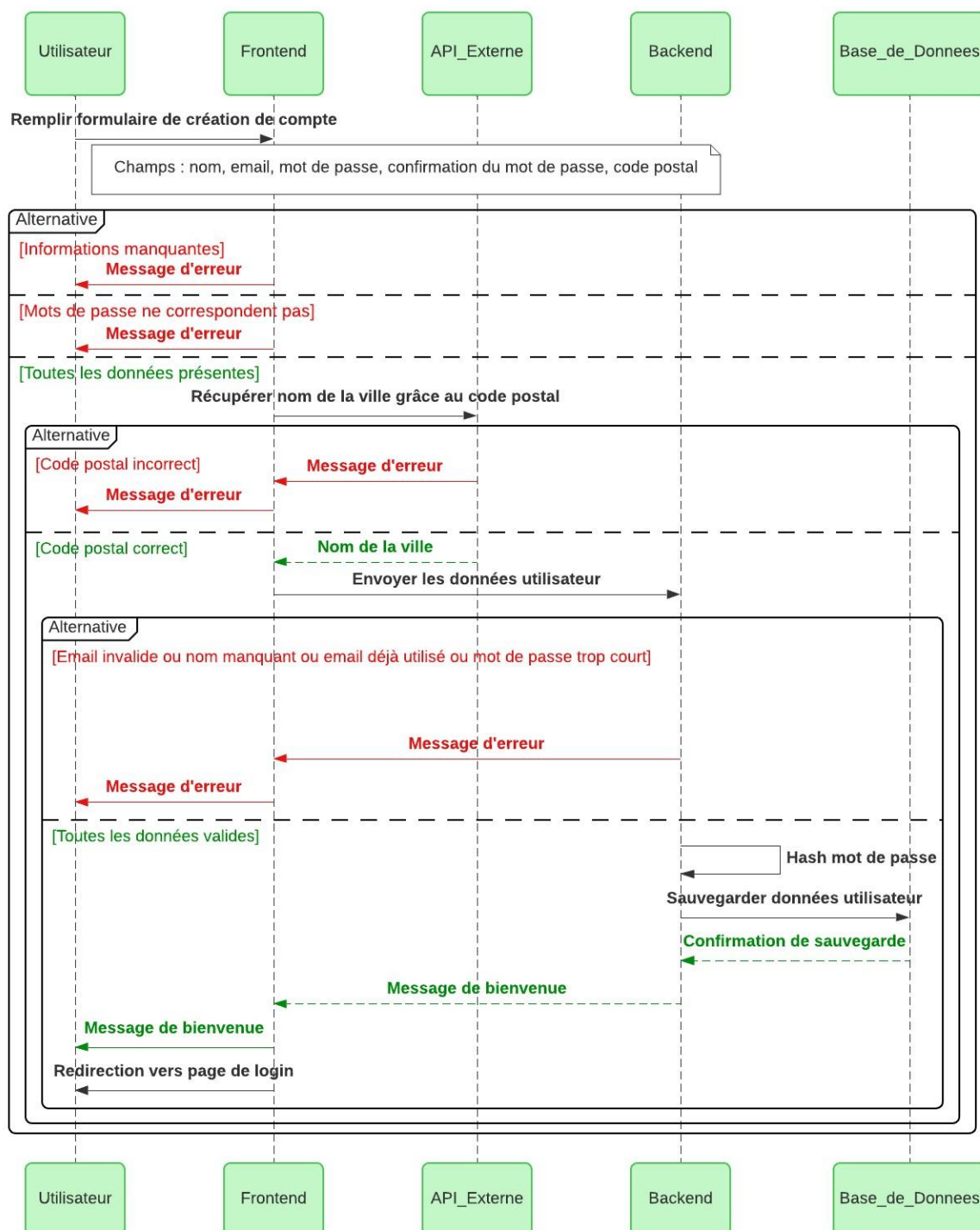
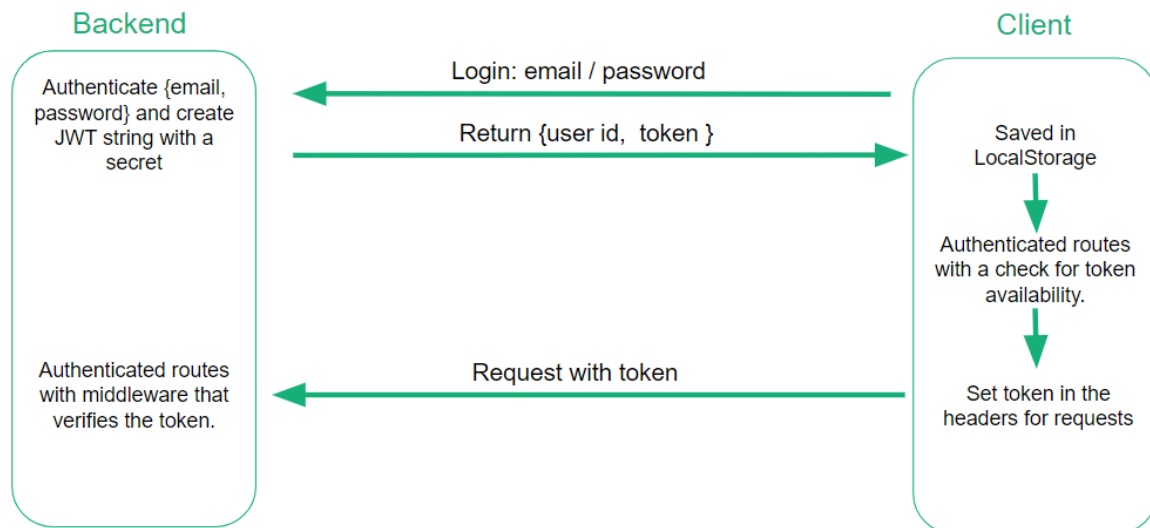


Figure 5-9 Diagramme de séquence - création de compte

5.3.4 Authentification via Json Web Token

J'ai également mis en place un système d'authentification via l'utilisation de Json Web Token.

Voir schéma ci-dessous :



Quand un utilisateur entre dans la page de « login » son identifiant et son mot de passe, le back-end génère et retourne au front-end un token contenant l'ID de l'utilisateur et une date d'expiration. Ce token est valide pendant 12 heures. Il est par la suite stocké dans le « local storage » du navigateur du client. Cela permet de renforcer la sécurité de l'application contre les attaques CSRF, mais nous verrons dans la partie 5.5 les risques liés à stocker le token dans le local storage.

Un middleware d'authentification a été mis en place côté front-end afin de vérifier la validité du token lors d'une requête pour les routes nécessitant une authentification.

- `"/createad"`: La page où un utilisateur peut créer un produit.
- `"/account"`: La page du compte d'un utilisateur.
- `"/account-annonces"`: La page où un utilisateur peut gérer ses produits.
- `"/user/:id/modify-ad/:id"`: La page où un utilisateur peut modifier les informations à propos d'un produit.

Voici le code du middleware d'authentification :

```

import { Navigate } from 'react-router-dom';
import PropTypes from 'prop-types';
import jwt_decode from 'jwt-decode';
import { accountService } from '../services/accountService';

const AuthenticatedRoute = ({ children }) => {
  const token = accountService.getToken();

  const isTokenExpired = () => {
    if (token) {
      const { exp } = jwt_decode(token);
      const expirationTime = (exp * 1000) - 60000;
      return Date.now() >= expirationTime;
    }
  }
}
  
```



```

    return true;
  };

  if (isTokenExpired()) {
    if (accountService.isLogged()) {
      accountService.logout();
      return <Navigate to="/login" />;
    }
    return <Navigate to="/login" />;
  }

  return children;
};

AuthenticatedRoute.propTypes = {
  children: PropTypes.node.isRequired,
};

export default AuthenticatedRoute;

```

Lors de l'exécution de la requête le token est renvoyé au back-end qui contrôle que le token est valide via également la mise en place d'un middleware d'authentification pour les routes suivantes nécessitant une authentification.

- Routes liées aux utilisateurs :
 - **GET api/users/:id** - Renvoie les informations d'un utilisateur avec l'ID spécifié.
 - **GET /api/users/:id/products** - Renvoie tous les produits publiés par un utilisateur.
 - **PUT api/users/:id** - Met à jour les informations d'un utilisateur avec l'ID spécifié.
 - **DELETE api/users/:id** - Supprime l'utilisateur avec l'ID spécifié et tous les produits le concernant.
- Routes liées aux produits :
 - **POST api/products** - Crée un nouveau produit.
 - **PUT api/products/:id** - Met à jour le produit avec l'ID spécifié.
 - **DELETE api/products/:id** - Supprime le produit avec l'ID spécifié.

Voici le code du middleware d'authentification côté back-end.

```

const auth = (req, res, next) => {
  try {
    const token = req.headers.authorization.split(' ')[1];
    jwt.verify(token, process.env.SECRET_TOKEN, (err) => {
      if (err) {
        return res.status(401).json({ message: 'Merci de vous authentifier.' });
      }
      next();
    });
  } catch (error) {
    res.status(401).json({ message: 'Merci de vous authentifier.' });
  }
};

module.exports = auth;

```



5.4 Présentation des tests

Je vais présenter des tests « end to end » permettant de vérifier le bon fonctionnement des API de la création à la suppression d'un utilisateur.

Pour ces tests j'ai utilisé les bibliothèques « Chai » et « chai-http ».

Avant de commencer les tests ainsi qu'à la fin des tests, je m'assure que l'ensemble des utilisateurs sont bien supprimés de la base de données.

```
const chai = require('chai');
const chaiHttp = require('chai-http');
const server = require('../server');
const User = require('../models/User');
var expect = require('chai').expect
var should = require('chai').should();

chai.use(chaiHttp);

before((done) => {
  User.deleteMany()
    .then(() => {
      console.log('Cleaned database');
      done();
    });
})

after((done) => {
  User.deleteMany()
    .then((result) => {
      console.log(result);
      done();
    });
})
```

Le 1^{er} test doit vérifier qu'il y a bien zéro utilisateur dans la base de données.

```
it('should verify that we have 0 user in the DB', (done) => {
  chai.request(server)
    .get('/api/user')
    .end((err, res) => {
      res.should.have.status(200);
      expect(res.body).to.be.an('object');
      expect(res.body.user.length).to.be.eql(0);
      done();
    });
});
```

Le 2^e test doit retourner une erreur si le mot de passe n'a pas au moins 6 caractères.

```
it('should test the password length', (done) => {

  const user = {
    "userName": "Maurice",
    "email": "maurice@gmail.com",
    "password": "123",
    "adress": {
      "street": "",
      "city": "Paris",
      "postalCode": "75010",
      "country": ""
    }
  }
```



```
}
chai.request(server)
  .post('/api/auth/signup')
  .send(user)
  .end((err, res) => {
    res.should.have.status(400);
    const actualMessage = res.body.message;
    expect(actualMessage).to.be.equal('Le mot de passe doit avoir au moins 6 caractères.');
```

Le 3^e test vérifie la création d'un utilisateur avec succès.

```
it('should POST a valid user', (done) => {

  const user = {
    "userName": "Maurice",
    "email": "maurice@gmail.com",
    "password": "123456",
    "adress": {
      "street": "",
      "city": "Paris",
      "postalCode": "75010",
      "country": ""
    }
  }
  chai.request(server)
    .post('/api/auth/signup')
    .send(user)
    .end((err, res) => {
      res.should.have.status(201);
      const actualMessage = res.body.message;
      expect(actualMessage).to.be.equal('Bienvenue Maurice');
```

Le 4^e test doit retourner une erreur si l'email d'un utilisateur existe déjà lors de l'enregistrement.

```
it('test if user email already exist', (done) => {

  const user = {
    "userName": "Maurice",
    "email": "maurice@gmail.com",
    "password": "123456",
    "adress": {
      "street": "",
      "city": "Paris",
      "postalCode": "75010",
      "country": ""
    }
  }
  chai.request(server)
    .post('/api/auth/signup')
    .send(user)
    .end((err, res) => {
      res.should.have.status(409);
      const actualMessage = res.body.message;
      expect(actualMessage).to.be.equal('Cet e-mail est déjà utilisé.');
```



Le 5^e test vérifie qu'un utilisateur peut se connecter et mettre à jour ses informations personnelles.

```
it('test the login', (done) => {  
  const login = {  
    "email": "maurice@gmail.com",  
    "password": "123456"  
  }  
  chai.request(server)  
    .post('/api/auth/login')  
    .send(login)  
    .end((err, res) => {  
      console.log('this runs the login part');  
      res.body.should.be.an('object');  
      token = res.body.token;  
      userId = res.body.userId;  
  
      const userUpdate = {  
        "userName": "Jean",  
        "email": "maurice@gmail.com",  
        "password": "123456",  
        "adress": {  
          "street": "10 rue de la tour",  
          "city": "Paris",  
          "postalCode": "75010",  
          "country": "France"  
        }  
      }  
    })  
    .put(`/api/user/${userId}`)  
    .set("Authorization", `Bearer ${token}`)  
    .send(userUpdate)  
    .end((err, res) => {  
      console.log('this runs the update user part');  
      res.should.have.status(200);  
      const actualMessage = res.body.message;  
      expect(actualMessage).to.be.equal('Profil mis à jour avec succès');  
      done();  
    });  
});  
});
```

Le 6^e test vérifie qu'il y a bien un utilisateur dans la base de données.

```
it('should verify that we have 1 user in the DB', (done) => {  
  chai.request(server)  
    .get('/api/user')  
    .end((err, res) => {  
      res.should.have.status(200);  
      res.body.should.be.an('object');  
      res.body.user.length.should.be.eql(1);  
      done();  
    });  
});
```

Le 7^e test doit supprimer l'utilisateur de la base de données.

```
it('should delete the user in the DB', (done) => {  
  chai.request(server)  
    .delete(`/api/user/${userId}`)
```



```
.set("Authorization", `Bearer ${token}`)
.end((err, res) => {
  res.should.have.status(200);
  const actualMessage = res.body.message;
  expect(actualMessage).to.be.equal('User deleted !');
  done();
});
});
```

Le dernier test vérifie qu'il y a bien zero utilisateur dans la base de données.

```
it('should verify that we have 0 user in the DB', (done) => {
  chai.request(server)
    .get('/api/user')
    .end((err, res) => {
      res.should.have.status(200);
      expect(res.body).to.be.an('object');
      expect(res.body.user.length).to.be.eql(0);
      done();
    });
});
})
```



5.5 Description de la veille, vulnérabilités de sécurité

Si je devais refaire mon application aujourd'hui, j'utiliserais Typescript pour typer les données dans le code. Cela permet de détecter les erreurs de type au moment de la compilation, avant même que le code ne soit exécuté. De plus, cela permet de mieux contrôler les entrées utilisateur et garantir qu'elles respectent les formats attendus. Cela réduit notamment le risque d'injections SQL, XSS (Cross-Site Scripting) et d'autres attaques basées sur des entrées malveillantes.

Lors de l'authentification d'un utilisateur, les tokens sont sauvegardées dans le « local storage » du client ce qui représente une vulnérabilité aux attaques XSS. Si un attaquant parvient à injecter du code malveillant dans l'application, il peut accéder aux tokens stockés et les utiliser pour usurper l'identité de l'utilisateur. Une meilleure pratique serait de stocker les JWT dans un cookie sécurisé (HttpOnly).

Pour les attaques DDoS (Distributed Denial of Service), il n'y a actuellement rien de mis en place pour les éviter. Cependant je pourrais limiter ce risque en utilisant un middleware « express-rate-limit » afin de limiter le nombre de requêtes pour une IP sur une période de temps défini.

Comme cela fait plusieurs mois que j'ai réalisé ce projet, il est actuellement nécessaire de mettre à jour les dépendances du projet. J'ai des alertes de vulnérabilité qui sont remontées par Dependabot sur Github.

6 Conclusion

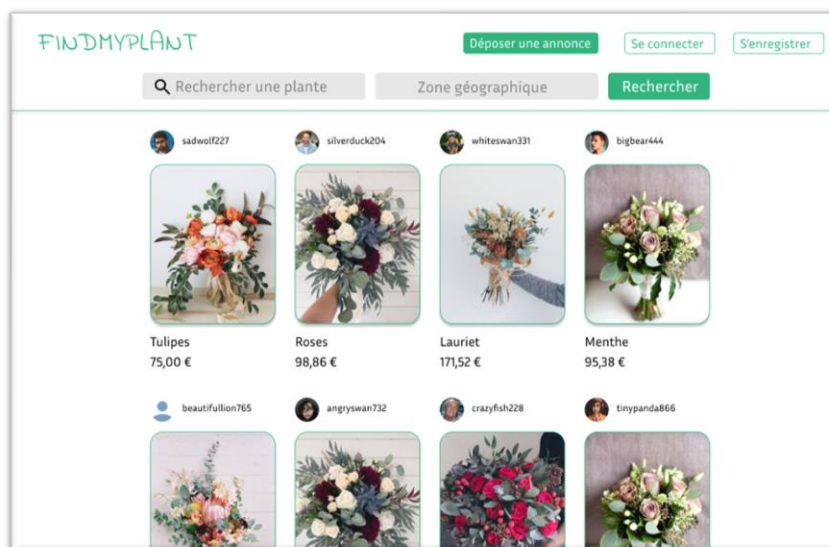
Ce projet m'a appris énormément sur le développement web. Le fait d'avoir travaillé sur le back-end et le front-end m'a permis de mieux comprendre leurs fonctionnements et comment les faire communiquer entre eux. Toutes les problématiques que j'ai pu rencontrer m'ont donné l'occasion d'en apprendre davantage.

Une bonne organisation a été primordiale pour mener à bien ce projet dans le temps imparti. Découper le projet en plusieurs petites tâches permet de mieux évaluer le temps nécessaire pour leurs réalisations.

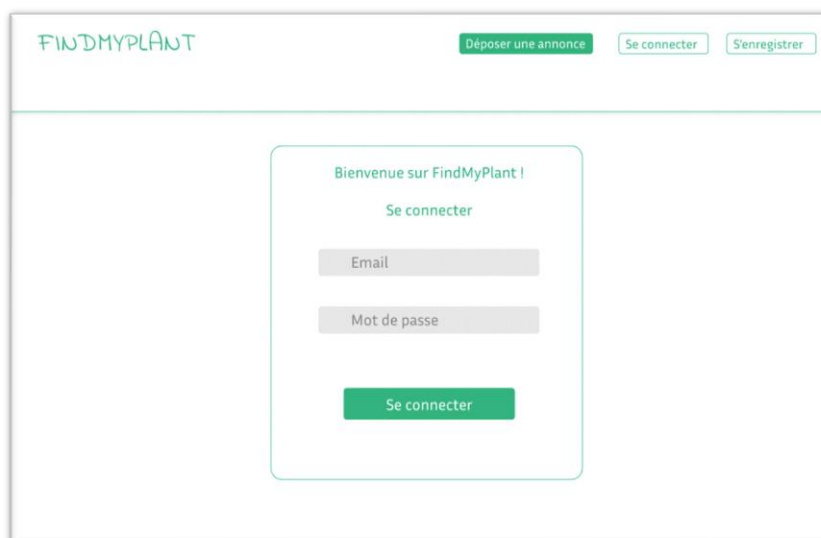


7 Annexes

Annexe 1 : Maquette **Page d'accueil**



Annexe 2 : Maquette **Page de Connexion**



Annexe 3 : Maquette **Page S'enregistrer**

Maquette de la page S'enregistrer. Le header contient le logo FINDMYPLANT, un bouton Déposer une annonce, et des boutons Se connecter et S'enregistrer. Le contenu principal est un formulaire intitulé 'Rejoins les passionnés de plantes sur FindMyPlant !' avec le sous-titre 'Créer un compte'. Le formulaire comprend des champs pour le Nom de l'utilisateur, l'Email, le Mot de passe, la Confirmation du Mot de passe, et le Code Postal, suivis d'un bouton Continuer.

Annexe 4 : Maquette **Page Déposer une Annonce**

Maquette de la page Déposer une Annonce. Le header contient le logo FINDMYPLANT, un bouton Déposer une annonce, et des boutons S'enregistrer, Mon compte, et Déconnexion. Le contenu principal est un formulaire intitulé 'Déposer une annonce'. Le formulaire comprend des champs pour le Nom de la plante (obligatoire), une sélection pour le type d'annonce (Je vends, Je donne, Je troque - obligatoire), un champ pour le Prix (exemple : 10 euros), un champ pour le Commentaire, un bouton Ajouter une image, et un bouton Créer.

Annexe 5 : Maquette **Page Mon compte**

FINDMYPLANT [Déposer une annonce](#) **Mon espace**
Mes informations
Mes annonces
Déconnexion

Mes informations

Nom d'utilisateur:

Email:

Mot de passe:

Adresse:

rue:

Ville:

Code postal:

Pays:

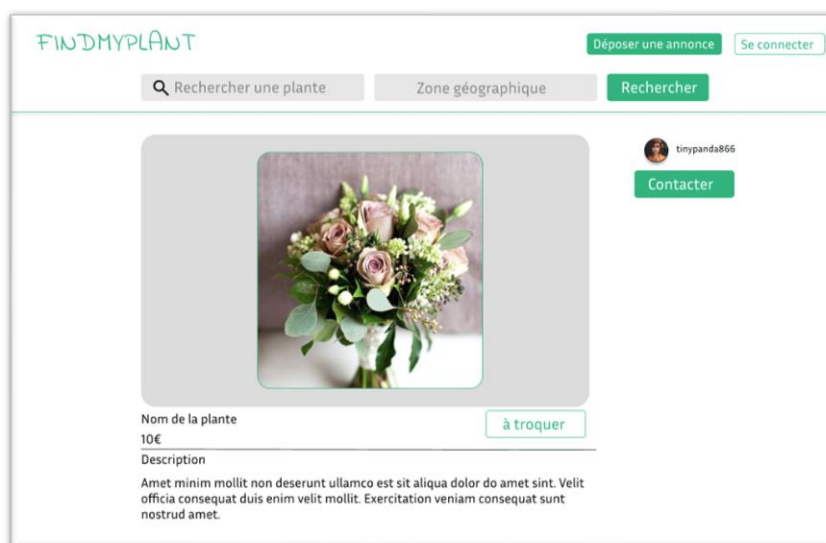
[Modifier](#) [Confirmer](#) [Supprimer](#)

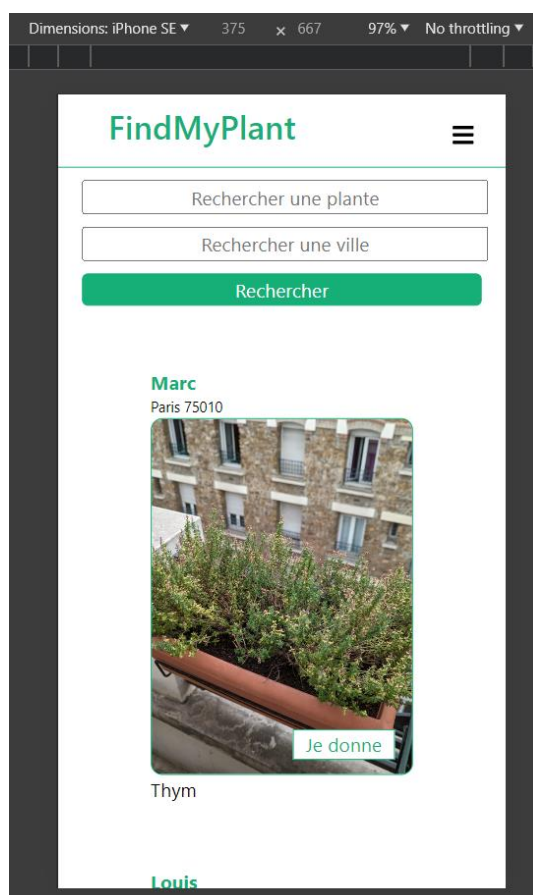
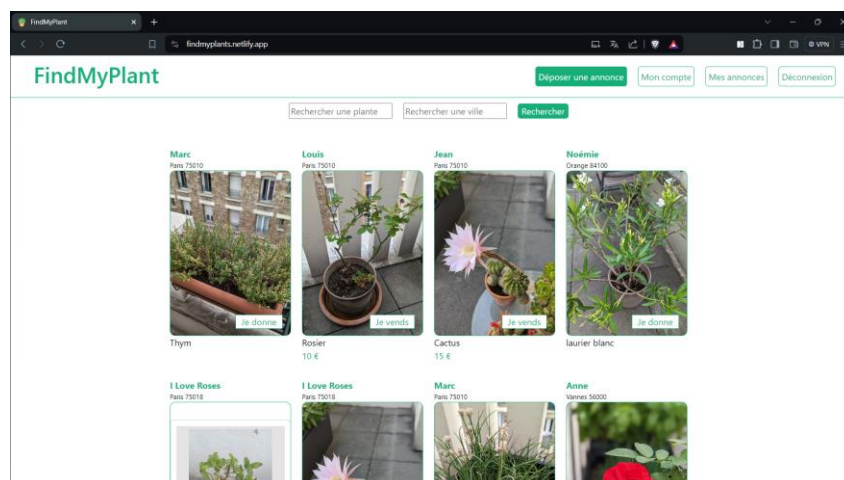
Annexe 6 : Maquette **Page Mes Annonces**

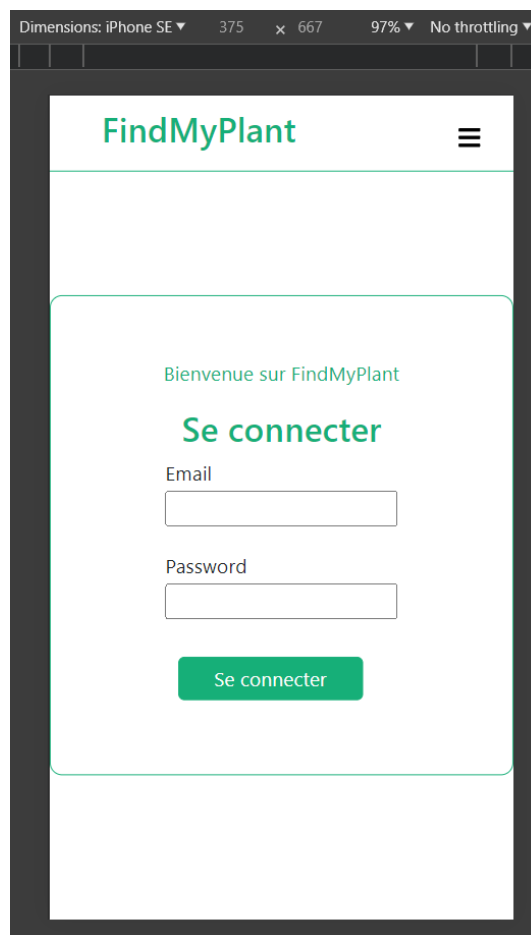
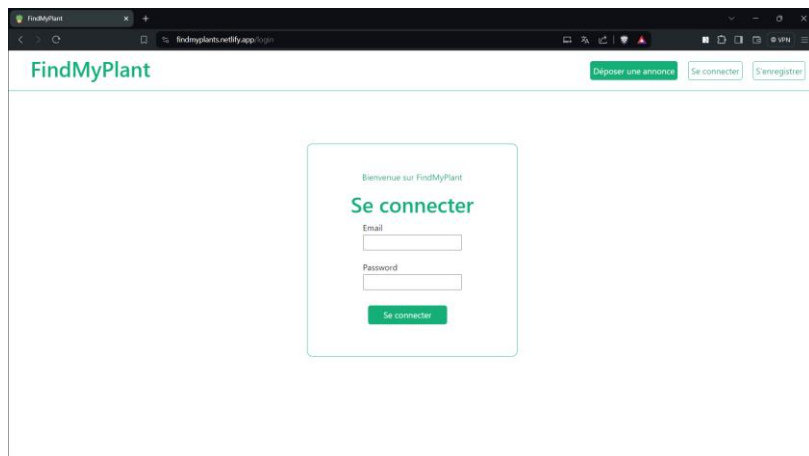
FINDMYPLANT [Déposer une annonce](#) **Mon espace**
Mes informations
Mes annonces
Déconnexion

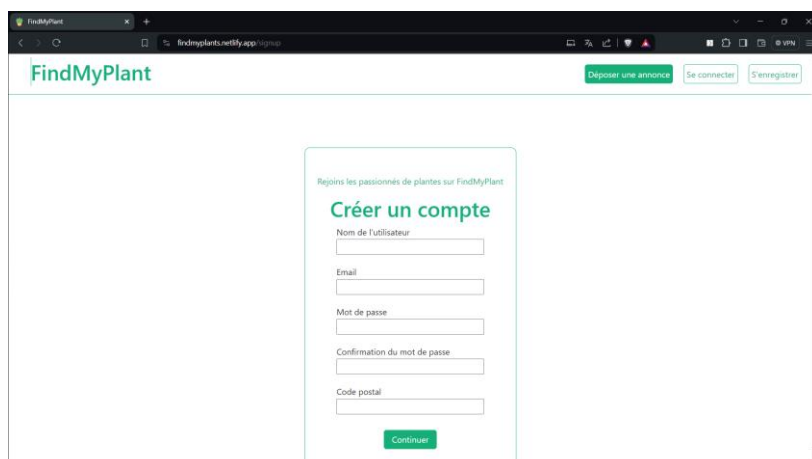
Mes annonces

			Disponibilité
	Rose Prix: 10€ Date de mise en ligne : 02/06/2023	Modifier Supprimer	☒
	Menthe à donner Date de mise en ligne : 02/06/2023	Modifier Supprimer	☒
	Ciboulette à troquer Date de mise en ligne : 02/06/2023	Modifier Supprimer	☒

Annexe 7 : Maquette **Page de Détails de la Plante**

Annexe 8 : Capture d'écran **Page d'accueil**

Annexe 9 : Capture d'écran **Page de Connexion**

Annexe 10 : Capture d'écran **Page S'enregistrer**

FindMyPlant

Rejoins les passionnés de plantes sur FindMyPlant

Créer un compte

Nom de l'utilisateur

Email

Mot de passe

Confirmation du mot de passe

Code postal

Continuer

Deposer une annonce Se connecter S'enregistrer



Dimensions: iPhone SE 375 x 667 97% No throttling

FindMyPlant

Rejoins les passionnés de plantes sur FindMyPlant

Créer un compte

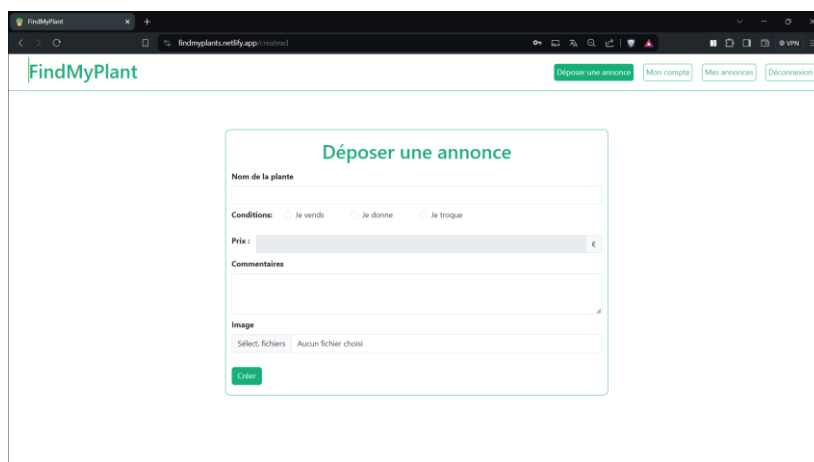
Nom de l'utilisateur

Email

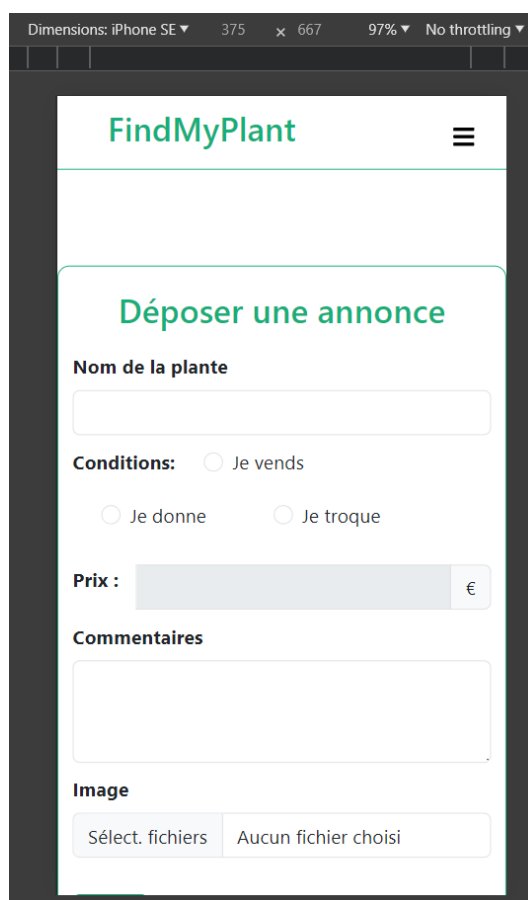
Mot de passe

Confirmation du mot de passe

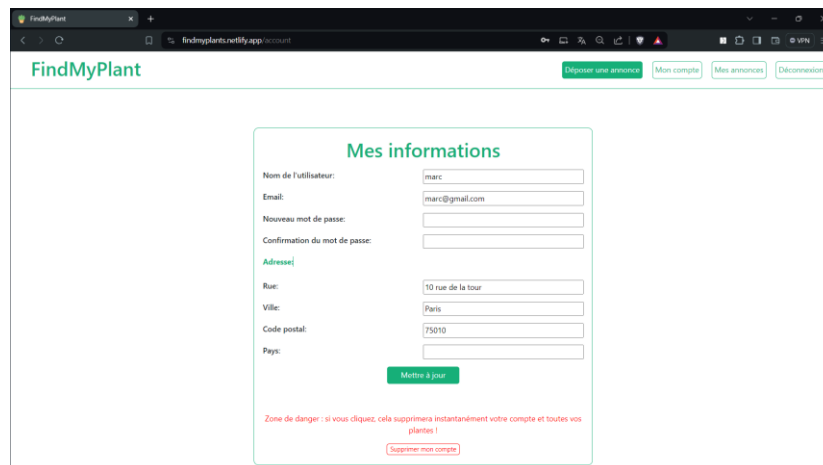
Code postal

Annexe 11 : Capture d'écran **Page Déposer une Annonce**

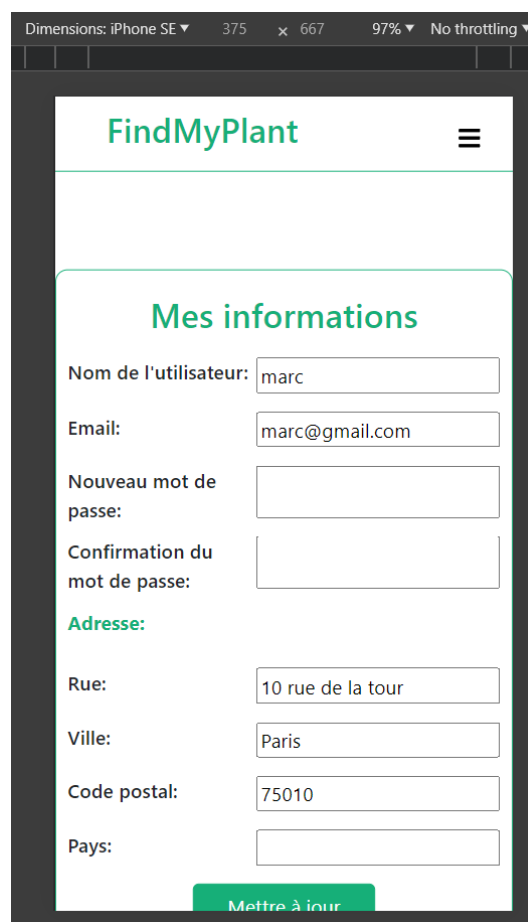
The screenshot shows a web browser window with the URL `findmyplant.netlify.app`. The page title is "FindMyPlant". In the top right corner, there are links: "Déposer une annonce" (highlighted in green), "Mon compte", "Mes annonces", and "Déconnexion". The main content is a form titled "Déposer une annonce". The form fields are: "Nom de la plante" (text input), "Conditions:" with radio buttons for "Je vends", "Je donne", and "Je troque", "Prix:" (text input with a Euro symbol), "Commentaires" (text area), and "Image" (file upload area with "Sélect. fichiers" and "Aucun fichier choisi" buttons). A green "Créer" button is at the bottom of the form.



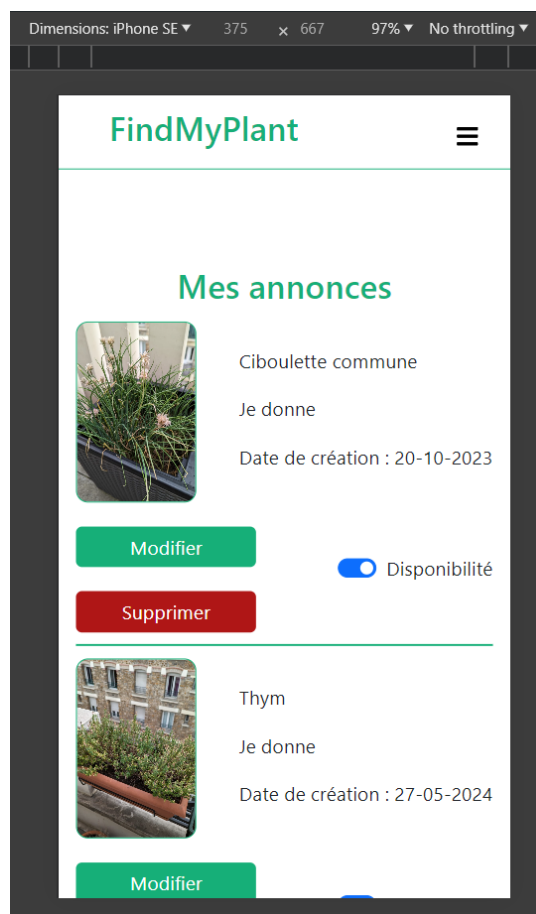
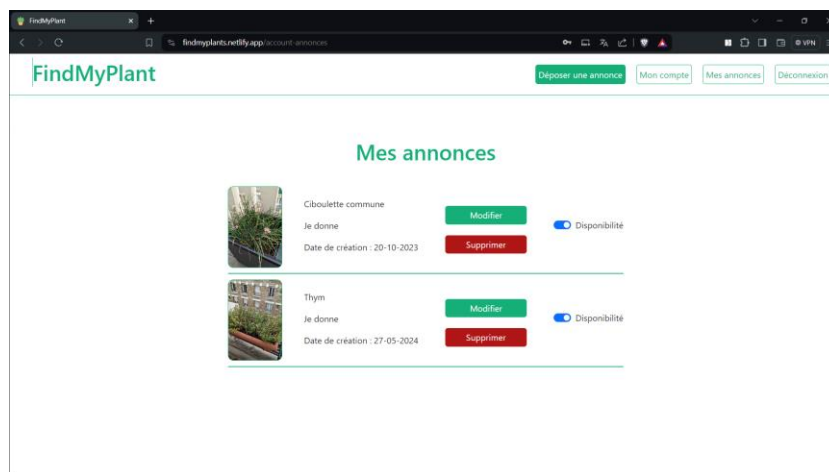
The screenshot shows a mobile app interface for an iPhone SE. The dimensions are 375 x 667 pixels at 97% zoom. The app has a dark header with the "FindMyPlant" logo and a hamburger menu icon. The form titled "Déposer une annonce" is centered on the screen. It contains the same fields as the web version: "Nom de la plante", "Conditions:" (radio buttons for "Je vends", "Je donne", "Je troque"), "Prix:" (text input with a Euro symbol), "Commentaires", and "Image" (file upload area with "Sélect. fichiers" and "Aucun fichier choisi" buttons). A green "Créer" button is at the bottom of the form.

Annexe 12 : Capture d'écran **Page Mon compte**

The screenshot shows the 'Mes informations' (My information) page on the FindMyPlant website. The page has a white background with a green border. At the top, there is a navigation bar with the FindMyPlant logo and links for 'Déposer une annonce', 'Mon compte', 'Mes annonces', and 'Déconnexion'. The main content area is titled 'Mes informations' in green. It contains several input fields for user information: 'Nom de l'utilisateur:' (marc), 'Email:' (marc@gmail.com), 'Nouveau mot de passe:', 'Confirmation du mot de passe:', 'Adresse:' (with sub-fields for Rue: 10 rue de la tour, Ville: Paris, Code postal: 75010, and Pays:), and a green 'Mettre à jour' button. At the bottom, there is a red warning message: 'Zone de danger : si vous cliquez, cela supprimera instantanément votre compte et toutes vos plantes !' and a red 'Supprimer mon compte' button.



The screenshot shows the 'Mes informations' (My information) page on the FindMyPlant mobile app. The page is displayed on an iPhone SE screen with dimensions 375 x 667 and 97% zoom. The app has a white background with a green border. At the top, there is a navigation bar with the FindMyPlant logo and a hamburger menu icon. The main content area is titled 'Mes informations' in green. It contains several input fields for user information: 'Nom de l'utilisateur:' (marc), 'Email:' (marc@gmail.com), 'Nouveau mot de passe:', 'Confirmation du mot de passe:', 'Adresse:' (with sub-fields for Rue: 10 rue de la tour, Ville: Paris, Code postal: 75010, and Pays:), and a green 'Mettre à jour' button.

Annexe 13 : Capture d'écran **Page Mes Annonces**

Annexe 14 : Capture d'écran **Page Modifier une annonce**

The desktop screenshot shows the 'Modifier une annonce' (Edit ad) page. The form includes a title field with 'Thym', a 'Valider' button, and radio buttons for 'Conditions' (Je vends, Je donne, Je troque). The 'Prix' field is set to 0. There is a 'Commentaires' text area and an 'Image' section with a file upload button. A 'Modifier' button is at the bottom.

FindMyPlant

Modifier une annonce

Nom de la plante

Thym

Valider

Conditions: ☐ Je vends ☒ Je donne ☐ Je troque

Prix : 0 €

Commentaires

Image

Sélect. fichiers Aucun fichier choisi

Modifier

The mobile screenshot shows the 'Modifier une annonce' (Edit ad) page. The form includes a title field with 'Ciboulette commune', a 'Valider' button, and radio buttons for 'Conditions' (Je vends, Je donne, Je troque). The 'Prix' field is set to 0. There is a 'Commentaires' text area with the text 'Je peux donner une partie de ma ciboulette.' and an 'Image' section with a file upload button.

FindMyPlant

Modifier une annonce

Nom de la plante

Ciboulette commune

Valider

Conditions: ☐ Je vends ☒ Je donne ☐ Je troque

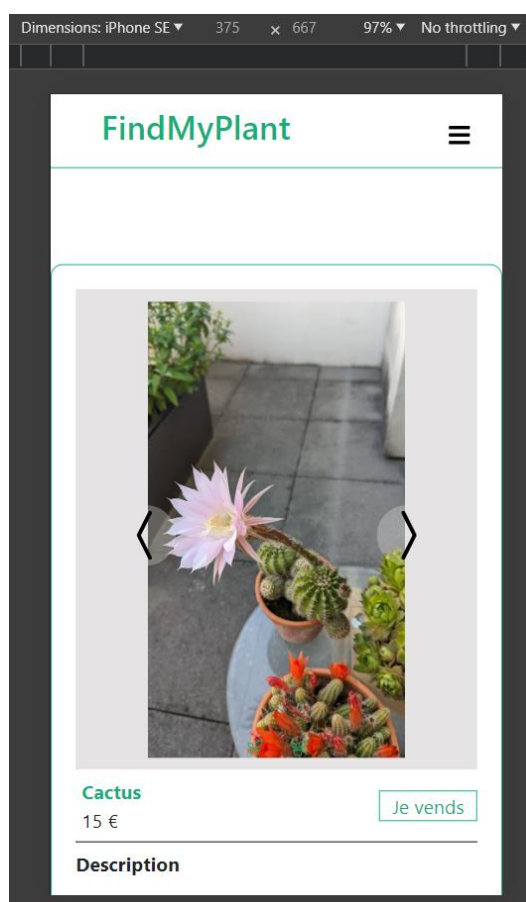
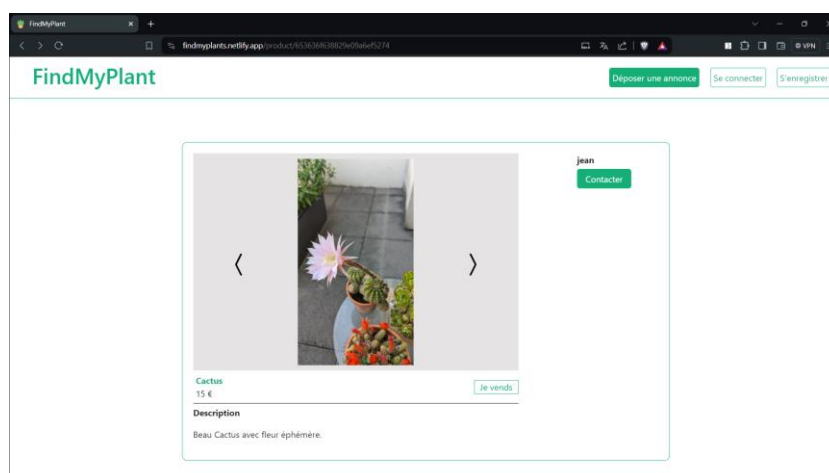
Prix : 0 €

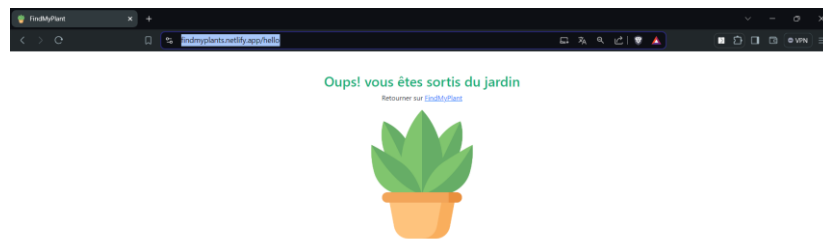
Commentaires

Je peux donner une partie de ma ciboulette.

Image

Sélect. fichiers Aucun fichier choisi

Annexe 15 : Capture d'écran **Page de Détails de la Plante**

Annexe 16 : Capture d'écran **Page 404**

Annexe 17: Front-end – composant « app »

```

import { BrowserRouter, Routes, Route } from 'react-router-dom';
import Home from '../views/Home';
import CreateAd from '../views/CreateAd';
import Login from '../views/Login';
import Signup from '../views/Register';
import AuthenticatedRoute from './AuthenticatedRoute';
import Account from '../views/Account';
import ProductDetail from '../views/ProductDetail';
import AccountProducts from '../views/AccountProducts';
import ModifyAd from '../views/ModifyAd';
import NotFound from '../views/NotFound';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/createad" element={
          <AuthenticatedRoute>
            <CreateAd />
          </AuthenticatedRoute>
        } />
        <Route path="/login" element={<Login />} />
        <Route path="/signup" element={<Signup />} />
        <Route path="/account" element={
          <AuthenticatedRoute>
            <Account />
          </AuthenticatedRoute>
        } />
        <Route path="/product/:id" element={<ProductDetail />} />
        <Route path="/account-annonces" element={
          <AuthenticatedRoute>
            <AccountProducts />
          </AuthenticatedRoute>
        } />
        <Route path="/user/:id/modify-ad/:id" element={
          <AuthenticatedRoute>
            <ModifyAd />
          </AuthenticatedRoute>
        } />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;

```

Annexe 18 : Front-end – Service de gestion du token

```

let saveToken = (token) => {
  localStorage.setItem('token', token);
};

let getToken = () => {
  return localStorage.getItem('token');
};

let saveUserId = (uid) => {
  localStorage.setItem('userId', uid);
};

let getUserId = () => {
  return localStorage.getItem('userId');
};

```



```

let logout = () => {
  localStorage.removeItem('token');
  localStorage.removeItem('userId');
};

let isLoggedIn = () => {
  let token = localStorage.getItem('token');

  return !!token;
};

export const accountService = {
  saveToken, getToken, logout, isLoggedIn, saveUserId, getUserId
};

```

Annexe 19: Front-end – Page d'accueil

```

import NavBar from '../components/NavBar';
import Products from '../components/Products';

function Home() {
  return (
    <div>
      <NavBar />
      <Products />
    </div>
  );
}

export default Home;

```

Annexe 20: Front-end – Page création d'une annonce

```

import axios from 'axios';
import NavBar from '../components/NavBar';
import Button from 'react-bootstrap/Button';
import Form from 'react-bootstrap/Form';
import 'bootstrap/dist/css/bootstrap.min.css';
import '../styles/CreateAd.css';
import Container from 'react-bootstrap/Container';
import { useState } from 'react';
// import { useNavigate } from 'react-router';
import InputGroup from 'react-bootstrap/InputGroup';
import SearchPlant from '../components/SearchPlant';
import { accountService } from '../services/accountService';

export default function CreateAd() {
  const apiUrl = process.env.REACT_APP_API_URL;
  const endpointproduct = process.env.REACT_APP_ENDPOINT_PRODUCTS;
  const uid = localStorage.getItem('userId');
  const [message, setMessage] = useState('');
  const [planteNameMessage, setPlanteNameMessage] = useState('');
  const [conditionMessage, setConditionMessage] = useState('');
  const [imageMessage, setImageMessage] = useState('');
  const [isLoading, setIsLoading] = useState(false);
  const [form, setForm] = useState({
    userId: uid,
    plantName: '',
    condition: '',
    price: '',
    comment: '',
    image: [],
  });
  const token = accountService.getToken();

```




```

// const navigate = useNavigate();

// These methods will update the state properties.
function updateForm(value) {
  return setForm((prev) => {
    return { ...prev, ...value };
  });
}

function handleImageUpload(e) {
  const file = e.target.files;
  updateForm({ image: file });
}

// This function will handle the submission.
async function onSubmit(e) {
  e.preventDefault();
  setIsLoading(true);

  const formData = new FormData();
  formData.append('userId', form.userId);
  formData.append('plantName', form.plantName);
  formData.append('condition', form.condition);
  formData.append('price', form.price);
  formData.append('comment', form.comment);
  for (let i = 0; i < form.image.length; i++) {
    formData.append('image', form.image[i]);
  }

  setPlanteNameMessage('');
  setConditionMessage('');
  setImageMessage('');
  setMessage('');

  await axios
    .post(`${apiUrl}${endpointproduct}`, formData, {
      headers: { Authorization: `Bearer ${token}` },
    })
    .then(() => {
      setIsLoading(false);
      console.log('Creation réussi !');
      setMessage('Annonce créée avec succès!');
    })
    .catch((error) => {
      setIsLoading(false);
      if (error.response.data.message === 'Merci d\'indiquer un nom à votre plante.') {
        setPlanteNameMessage(error.response.data.message);
      }
      if (error.response.data.message === 'Merci d\'indiquer une condition à votre
annonce.') {
        setConditionMessage(error.response.data.message);
      }
      if (error.response.data.message === 'Merci d\'intégrer une image à votre
annonce.') {
        setImageMessage(error.response.data.message);
      }
    });
}

return (
  <div>
    <NavBar />
    <Container className="createAd-wrapper">
      <div className="createdAd-title">
        <h1>Déposer une annonce</h1>
      </div>

      <Form onSubmit={onSubmit}>
        <SearchPlant searchTerm={form.plantName} handleSearch={updateForm} />
        {planteNameMessage && <p className="errorMessage">{planteNameMessage}</p>}
      </Form>
    </Container>
  </div>
)

```



```

    { /* Select condition */ }
    <Form.Group className="createAd-group">
      <Form.Label>Conditions: </Form.Label>
      <div className="form-check form-check-inline">
        <Form.Check
          inline
          label="Je vends"
          type="radio"
          name="conditionOptions"
          id="vendre"
          value="Je vends"
          checked={form.condition === 'Je vends'}
          onChange={(e) => updateForm({ condition: e.target.value })}
        />
      </div>
      <div className="form-check form-check-inline">
        <Form.Check
          inline
          label="Je donne"
          type="radio"
          name="conditionOptions"
          id="donner"
          value="Je donne"
          checked={form.condition === 'Je donne'}
          onChange={(e) => {
            updateForm({ condition: e.target.value });
            updateForm({ price: '0' });
          }}
        />
      </div>
      <div className="form-check form-check-inline">
        <Form.Check
          inline
          label="Je troque"
          type="radio"
          name="conditionOptions"
          id="troquer"
          value="Je troque"
          checked={form.condition === 'Je troque'}
          onChange={(e) => {
            updateForm({ condition: e.target.value });
            updateForm({ price: '0' });
          }}
        />
      </div>
    </Form.Group>
    {conditionMessage && <p className="errorMessage">{conditionMessage}</p>}

    { /* Price */ }
    <InputGroup className="createAd-group">
      <Form.Label className="text-price">Prix :</Form.Label>
      <Form.Control
        className="createAd-input"
        aria-label="Amount (to the nearest euro)"
        type="text"
        value={form.price}
        onChange={(e) => updateForm({ price: e.target.value })}
        disabled={form.condition !== 'Je vends'}
      />
      <InputGroup.Text>€</InputGroup.Text>
    </InputGroup>

    { /* Comments */ }
    <Form.Group controlId="comments" className="createAd-group">
      <Form.Label>Commentaires</Form.Label>
      <Form.Control
        as="textarea"
        rows={3}
        value={form.comment}
      />
    </Form.Group>
  </Form>

```



```

        onChange={(e) => updateForm({ comment: e.target.value })}
      />
    </Form.Group>

    <Form.Group controlId="image" className="createAd-group">
      <Form.Label>Image</Form.Label>
      <Form.Control
        className="createAd-input"
        type="file"
        accept="image/*"
        multiple
        onChange={handleImageUpload}
      />
    </Form.Group>
    {imageMessage && <p className="errorMessage">{imageMessage}</p>}
    <Form.Group controlId="submit" className="createAd-group">
      <Button variant="primary" type="submit" style={{ marginTop: '20px',
        backgroundColor: '#16AF78', borderColor: '#16AF78' }}>
        Créer
      </Button>
    </Form.Group>
    {isLoading && <p className="infoMessage">En cours de chargement</p>}
    {message && <p className="succesMessage">{message}</p>}
  </Form>
</Container>
</div>
);
}

```

Annexe 21: Front-end – Page Mes annonces

```

import '../styles/AccountProducts.css';
import NavBar from '../components/NavBar';
import { useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
import Form from 'react-bootstrap/Form';
import axios from 'axios';
import moment from 'moment';
import { accountService } from '../services/accountService';

function AccountProducts() {
  const apiUrl = process.env.REACT_APP_API_URL;
  const endpointproduct = process.env.REACT_APP_END_POINT_PRODUCTS;
  const endpointuser = process.env.REACT_APP_END_POINT_USER;
  const [data, setData] = useState([]);
  const uid = localStorage.getItem('userId');
  const token = accountService.getToken();

  useEffect(() => {
    axios
      .get(`${apiUrl}${endpointuser}${uid}/products`, {
        headers: { Authorization: `Bearer ${token}` },
      })
      .then((res) => {
        setData(res.data.products);
      })
      .catch((error) => console.log(error));
  }, [uid, apiUrl, endpointuser, token]);

  const handleDeleteProduct = (productId) => {
    axios
      .delete(`${apiUrl}${endpointproduct}${productId}`, {
        headers: { Authorization: `Bearer ${token}` },
      })
      .then((response) => {
        console.log(response);
      })
  }
}

```



```

        const updatedData = data.filter((product) => product._id !== productId);
        setData(updatedData);
    })
    .catch((error) => {
        console.error('Erreur lors de la suppression du produit :', error);
    });
});

const handleSwitchToggle = (productId) => {
    const updatedProduct = data.find((product) => product._id === productId);
    console.log('updatedProduct = ', updatedProduct);
    const updatedAvailability = !updatedProduct.status;
    console.log('updatedAvailability = ', updatedAvailability);

    axios
        .put(
            `${apiUrl}${endpointproduct}${productId}`,
            { status: updatedAvailability },
            {
                headers: { Authorization: `Bearer ${token}` },
            },
        )
        .then((response) => {
            console.log(response);
            const updatedData = data.map((product) => {
                if (product._id === productId) {
                    return { ...product, status: updatedAvailability };
                }
                return product;
            });
            setData(updatedData);
        })
        .catch((error) => {
            console.error(
                'Erreur lors de la mise à jour de la disponibilité du produit :',
                error,
            );
        });
});

return (
    <div>
        <NavBar />
        <div className="accountProducts-wrapper">
            <h1 className="title">Mes annonces</h1>
            <article className="accountProducts">
                {data.map(
                    (
                        { _id, imageUrl, plantName, price, condition, status, createAt },
                        index,
                    ) => (
                        <div className="accountProducts-details" key={`_${_id}-${index}`}>
                            <div className="accountProducts-mobile">
                                <div
                                    className="plant-cover"
                                    style={{ backgroundImage: `url(${imageUrl[0]})` }}
                                ></div>
                                <div className="accountProducts-details-text">
                                    <p>{plantName}</p>
                                    <p>{!price ? <p>{condition}</p> : <p>Prix : {price} €</p>}</p>
                                    <p>
                                        Date de création :{' '}
                                        {moment(createAt).format('DD-MM-YYYY')}
                                    </p>
                                </div>
                            </div>
                            <div className="accountProducts-mobile">
                                <div className="accountProducts-button">
                                    <button className="modify">

```



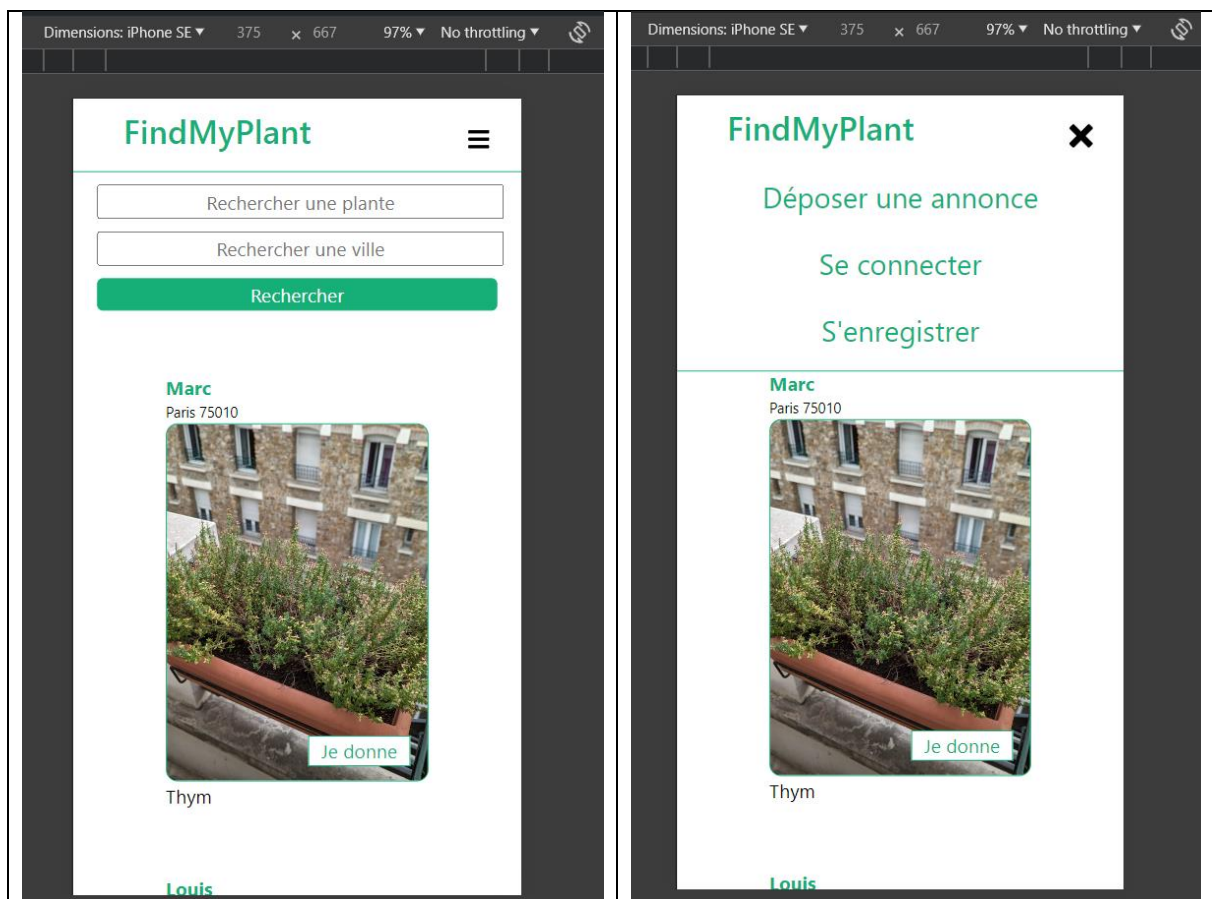
```

        <Link to={` /user/${uid}/modify-ad/${_id}`} key={_id}>
          Modifier
        </Link>
      </button>
      <button
        className="delete"
        onClick={() => handleDeleteProduct(_id)}
      >
        Supprimer
      </button>
    </div>
    <Form>
      <Form.Check
        type="switch"
        checked={status}
        id={_id}
        label="Disponibilité"
        onChange={() => handleSwitchToggle(_id)}
      />
    </Form>
  </div>
</div>
),
})
</article>
</div>
</div>
);
}

export default AccountProducts;

```

Annexe 22 : Front-end – Responsive Design (Barre de navigation)



Annexe 23 : Back-end – Serveur Express

```
const express = require('express');
const mongoose = require('mongoose');
require('dotenv').config();

const morgan = require('morgan');
const router = require('./routes/index');

const app = express();
const PORT = process.env.PORT || 5000;
const DB_URL = process.env.NODE_ENV === 'production' ? process.env.MONGODB_URI_PROD :
process.env.MONGODB_URI_DEV;
const HTTP_ORIGIN = process.env.NODE_ENV === 'production' ? process.env.HTTP_ORIGIN_PROD :
process.env.HTTP_ORIGIN_DEV;

app.use((req, res, next) => {
  res.setHeader('Access-Control-Allow-Origin', `${HTTP_ORIGIN}`);
  res.setHeader('Access-Control-Allow-Headers', 'Origin, Content-Type, Authorization');
  res.setHeader('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE');
  next();
});

mongoose.connect(DB_URL)
  .then(() => console.log(`Connexion à MongoDB ${process.env.NODE_ENV} réussie !`))
  .catch(() => console.log(`Echec de la connexion à MongoDB ${process.env.NODE_ENV} !`));

app.use(express.json());
app.use(morgan('dev'));

app.use('/', router);

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
module.exports = app;
```

Annexe 24 : Back-end – Schéma utilisateur (Mongoose)

```
const mongoose = require('mongoose');
const uniqueValidator = require('mongoose-unique-validator');

const { Schema } = mongoose;

const userSchema = new Schema({
  userName: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  adress: {
    street: { type: String },
    city: { type: String, required: false },
    postalCode: { type: String, required: false },
    country: { type: String, required: false },
  },
  profilPicture: { type: String },
});

userSchema.plugin(uniqueValidator);

module.exports = mongoose.model('User', userSchema);
```



Annexe 25 : Back-end – Schéma produit (Mongoose)

```
const mongoose = require('mongoose');

const { Schema } = mongoose;

const productSchema = new Schema({
  plantName: { type: String, required: true },
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  condition: { type: String, required: true },
  price: { type: Number },
  comment: { type: String },
  createdAt: { type: Date, default: Date.now, required: true },
  status: { type: Boolean, default: true, require: true },
  imageUrl: { type: [String] },
});

module.exports = mongoose.model('Product', productSchema);
```

Annexe 26 : Back-end – Middleware de pagination

```
function paginatedResults(model) {
  return async (req, res, next) => {
    const page = (parseInt(req.query.page, 10) >= 1 ? parseInt(req.query.page, 10) : 1) || 1;
    const backendLimit = 12;
    const limit = parseInt(req.query.limit, 10) <= backendLimit ? req.query.limit : backendLimit;
    const { name, city } = req.query;
    const startIndex = (page - 1) * limit;
    const endIndex = page * limit;

    const results = {};

    results.backendLimit = backendLimit;

    try {
      const query = { status: true };

      if (name) {
        query.plantName = { $regex: name, $options: 'i' };
      }

      const products = await model.find(query)
        .sort({ createdAt: 'desc' })
        .populate('userId', '-email -password');

      const filteredProduct = products.filter((item) => {
        if (city) {
          return item.userId.adress.city.toLowerCase().includes(city.toLowerCase());
        }
        return true;
      });

      const sortedAndPaginatedProducts = filteredProduct
        .slice(startIndex, endIndex);

      results.totalProducts = filteredProduct.length;

      results.product = sortedAndPaginatedProducts;

      res.resultsPaginatedAndFiltered = results;
      next();
    } catch (error) {
      res.status(500).json({ message: error.message });
    }
  };
}
```



```
module.exports = paginatedResults;
```

Annexe 27 : Back-end – Middleware de gestion des images (Multer)

```
const multer = require('multer');

const storage = multer.diskStorage({
  destination: (req, file, callback) => {
    callback(null, 'images');
  },
  filename: (req, file, callback) => {
    const name = file.originalname.split(' ').join('_');
    callback(null, `${Date.now()}-${name}`);
  },
});

module.exports = multer({ storage });
```

Annexe 28 : Back-end – Création d'un produit (annonce)

```
static async createOneProduct(req, res) {
  const { userId, plantName, condition } = req.body;

  // Check the userId
  const foundUser = await User.findOne({ _id: userId });
  if (!foundUser) {
    return res.status(400).json({ message: 'L'utilisateur n'existe pas' });
  }

  // Check the plantName
  if (!plantName) {
    return res.status(400).json({ message: 'Merci d'indiquer un nom à votre plante.' });
  }

  // Check the condition
  if (!condition) {
    return res.status(400).json({ message: 'Merci d'indiquer une condition à votre annonce.' });
  }
};

// Check the image.
const listImage = req.files;
if (listImage.length === 0) {
  return res.status(400).json({ message: 'Merci d'intégrer une image à votre annonce.' });
}

try {
  const uploadPromises = listImage.map(async (image) => {
    // Save the image in Cloudinary
    const result = await cloudinary.uploader.upload(image.path, {
      folder: 'products',
      width: 600,
      crop: 'scale',
    });

    // Once saved, delete the image from the server.
    fs.unlink(image.path, (error) => {
      if (error) console.log(error);
      else {
        console.log(`${image.path} deleted`);
      }
    });
  });

  return result.secure_url;
```




```

    });

    const imageUrl = await Promise.all(uploadPromises);

    // Create and save new product in the DB.
    const product = new Product({
      ...req.body,
      imageUrl: imageUrl,
    });
    await product.save();

    return res.status(201).json({ product });
  } catch (error) {
    return res.status(500).json({ message: error.message });
  }
}

```

Annexe 29 : Back-end – Suppression d'un compte utilisateur

```

static async deleteUser(req, res) {
  try {
    // Find the user by ID
    const user = await User.findOne({ _id: req.params.id });
    if (!user) {
      return res.status(404).json({ message: 'Utilisateur introuvable' });
    }

    const products = await Product.find({ userId: user._id });
    if (products.length > 0) {
      products.forEach(async (product) => {
        await product.imageUrl.forEach(async (image) => {
          const publicId = cloudinaryPublicId(image);
          console.log(publicId);

          const result = await cloudinary.uploader.destroy(publicId);
          console.log(result);
          if (!result) {
            return res.status(500).json({ message: 'Une erreur s\'est produite lors de la suppression de l\'image' });
          }
          console.log('Image supprimé de Cloudinary');
        });
      });
      const deletedProducts = await Product.deleteMany({ userId: user._id });
      console.log(deletedProducts);
    }
    User.deleteOne({ _id: req.params.id })
      .then(() => res.status(200).json({ message: 'User deleted !' }))
      .catch((error) => res.status(400).json({ message: error.message }));
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
}

```



8 Table des illustrations

Figure 3-1 Diagramme des cas d'utilisations.....	6
Figure 3-2 Organisation et planning avec Trello	8
Figure 4-1: Architecture et technologie	8
Figure 5-1 Schéma d'enchaînement des maquettes.....	11
Figure 5-2 Schéma de représentation des composants React.....	13
Figure 5-3 Barre de recherche avec liste déroulante	14
Figure 5-4 Flowchart des routes front-end	15
Figure 5-5 Front-end - déploiement sur Netlify	17
Figure 5-6 Schéma de la base de données	19
Figure 5-7 Diagramme de classe UML (Routes et Contrôleurs)	20
Figure 5-8 Documentation API dans Postman (Modification des données d'un utilisateur) ..	23
Figure 5-9 Diagramme de séquence - création de compte.....	25